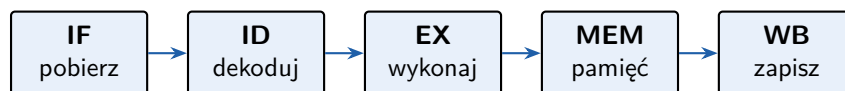


NOTATKI DO KURSU

Własny procesor RISC-V w SystemVerilogu

Od bramek logicznych do 5-etapowego potoku,
na którym uruchamiasz programy w C



Spis treści

Wstęp	1
0 Narzędzia i pierwszy układ	3
0.1 HDL to nie jest język programowania	3
0.2 Dwa rodzaje logiki	4
0.3 Anatomia modułu	4
0.4 Anatomia testbencha	5
0.5 Logika czterowartościowa: 0, 1, X, Z	6
0.6 Jak symulator wykonuje kod	6
0.7 GTKWave w minutę	6
0.8 Zadania	6
Pytania kontrolne	7
1 Logika kombinacyjna	8
1.1 assign i always_comb	8
1.2 Wektory: wycinanie, sklejanie, powielanie	8
1.3 Liczby ze znakiem w sprzęcie	9
1.4 Szerokości wyrażeń: najczęstsze źródło błędów	9
1.5 Jaki sprzęt powstaje	10
1.6 Przykład: sumator ripple-carry	10
1.7 if kontra case i priorytety	10
1.8 Zatrzaski: błąd numer jeden	11
1.9 Pętle for i generate: kopiowanie sprzętu	11
1.10 Przykład: przesuwnik logarytmiczny	12
1.11 Znane ograniczenia Icarus Verilog 12	12
1.12 Zadania	12
Pytania kontrolne	13
2 Logika sekwencyjna i automaty stanów	14
2.1 Przerzutnik i always_ff	14
2.2 Czas w układzie synchronicznym	15
2.3 Pamięci	16
2.4 Automaty skończone (FSM)	16
2.5 Porównania z parametrami	17
2.6 Kolejka FIFO: wskaźniki z dodatkowym bitem	18
2.7 Sygnały z zewnątrz (na przyszłość)	18
2.8 Zadania	18
Pytania kontrolne	19
3 ALU, bank rejestrów, komparator skoków	20
3.1 RV32I w pigułce	20
3.2 ALU	20

3.3	Bank rejestrów	21
3.4	Komparator skoków	22
3.5	Zadania	23
	Pytania kontrolne	23
4	Architektura RV32I i emulator w C	24
4.1	Formaty instrukcji	24
4.2	Stałe natychmiastowe (immediate)	24
4.3	Lista instrukcji RV32I	25
4.4	Pseudoinstrukcje i ładowanie stałych	25
4.5	Pamięć	26
4.6	Nasza maszyna	26
4.7	Asembler i disassembler kursu	26
4.8	Ślad wykonania	27
4.9	Emulator: struktura	27
4.10	Zadania	27
	Pytania kontrolne	28
5	Procesor jednocyklowy RV32I	29
5.1	Ścieżka danych	29
5.2	Przejście instrukcji przez ścieżkę danych	30
5.3	Tabela sterowania	30
5.4	LSU: bajty w słowie	31
5.5	Interfejs rdzenia: kontrakt z testbenchem	31
5.6	Jak debugować procesor	32
5.7	Programy testowe	33
5.8	Zadania	33
	Pytania kontrolne	33
6	Weryfikacja: co-symulacja, fuzzing, Verilator	35
6.1	Trzy poziomy testowania	35
6.2	Ograniczenia porównania śladów	36
6.3	Verilator	36
6.4	Mutacje: kto pilnuje strażnika?	36
6.5	Pokrycie funkcjonalne	37
6.6	Zadania	37
	Pytania kontrolne	37
7	Programy w C na Twoim procesorze	39
7.1	Toolchain	39
7.2	Od C do pliku .hex	39
7.3	Co się dzieje przed main()	40
7.4	Skrypt linkera	40
7.5	Brakujące instrukcje: __mulsi3 i spółka	40
7.6	MMIO i volatile	41
7.7	Konwencja wywołań (ilp32)	41
7.8	Zadania	42
	Pytania kontrolne	42
8	Procesor potokowy	43
8.1	Po co potok?	43
8.2	Etapy i rejestry potoku	44

8.3	Hazardy danych	44
8.4	Hazardy sterowania	46
8.5	Plan pracy	46
8.6	Zadania	46
	Pytania kontrolne	47
9	Co dalej: projekty rozszerzające	48
9.1	A. Rozszerzenie M (mnożenie i dzielenie)	48
9.2	B. CSR, wyjątki i przerwania	48
9.3	C. Procesor na FPGA	49
9.4	D. Pamięć z opóźnieniem i cache	49
9.5	E. Weryfikacja formalna	49
9.6	F. Oficjalne testy riscv-tests	49
9.7	Lektura	49
9.8	Prawdziwe rdzenie do przeczytania	50
A	Karta RV32I	51
B	Ściąga SystemVerilog	53
C	Ściąga poleceń	55
D	Słowniczek	56

Wstęp

O czym jest ten kurs

Celem kursu jest zbudowanie od zera procesora RISC-V (RV32I) w SystemVerilogu, najpierw w wersji jednocykowej, a potem jako klasyczny 5-etapowy potok, i uruchomienie na nim programów skompilowanych z C. Po drodze powstaje wszystko, czego potrzebuje projektant procesora: emulator referencyjny, infrastruktura testowa, co-symulacja i fuzzing.

Notatki zakładają, że znasz C i programowanie niskopoziomowe (wskaźniki, pamięć, ABI, assembler w zarysie). Nie zakładają znajomości języków opisu sprzętu ani elektroniki. Tam, gdzie intuicja programisty C prowadzi na manowce, znajdziesz czerwone ramki *Pułapka*.

Jak czytać te notatki

Każdy rozdział odpowiada jednemu modułowi kursu (katalogowi [NN-nazwa/](#)). Rozdział zaczyna się od metryczki (cel, czas, katalog), potem jest teoria z przykładami, zadania (zielone ramki) i pytania kontrolne. Rozwiązania zadań piszesz w plikach [rtl/*.sv](#) danego modułu, a gotowe testy w [tb/](#) mówią, czy działa.

Rozdz.	Moduł	Co budujesz	Wieczory
0	00-start	narzędzia, licznik, pierwszy testbench	1
1	01-kombinacyjna	mux, enkoder, sumator, barrel shifter; szerokości, zatrzaski	2
2	02-sekwencyjna	rejstry, timing, pamięci, FSM, FIFO, UART	2-3
3	03-alu-regfile	ALU, bank rejestrów, komparator skoków	1-2
4	04-isa-emulator	RV32I na poziomie bitów, generator immediate'ów, emulator w C	2-3
5	05-single-cycle	dekoder, LSU, procesor jednocyklowy	3-4
6	06-weryfikacja	co-symulacja, fuzzing, Verilator, mutacje	1-2
7	07-c-bare-metal	crt0, skrypt linkera, <code>__mulsi3</code> : C na Twoim CPU	1-2
8	08-pipeline	potok 5-etapowy: forwarding, stalle, flush	4-6
9	09-dalej	rozszerzenie M, przerwania, FPGA, cache, formalna	—

Moduły trzeba robić po kolei. Moduł 5 używa Twoich plików z modułów 3 i 4, moduł 6 używa emulatora z modułu 4, a moduł 8 używa wszystkiego.

Narzędzia

Narzędzie	Do czego
Icarus Verilog (iverilog, vvp)	symulacja zdarzeniowa, 4 wartości logiczne, główny symulator kursu
GTKWave	oglądanie przebiegów (pliki .vcd)
Verilator	lint (<i>-Wall dla sprzętu</i>) i szybka symulacja
Yosys	synteza: ile bramek, jak długa ścieżka krytyczna
rvtool.py	assembler i disassembler RV32I(M) dołączony do kursu
clang + ld.lld albo GCC dla RISC-V	tylko w rozdziale 7 (C na procesorze)

Do rozdziału 7 potrzebny jest linker: `sudo apt install lld` (albo pełny toolchain `gcc-riscv64-unknown-elf`). Pozostałe rozdziały nie wymagają żadnego toolchaina RISC-V.

Rytm pracy

W każdym module te same polecenia (uruchamiane z katalogu modułu):

```
make test           # wszystkie testy modułu (na starcie: same FAIL)
make unit T=mux4    # jeden test jednostkowy
make wave T=mux4    # przebiegi w GTKWave
make lint           # Verilator --lint-only
make synth T=adderr # Yosys: bramki i najdłuższa ścieżka
```

W modułach z procesorem (5–8) dochodzą:

```
make progs          # 12 programów testowych na Twoim rdzeniu
make prog P=08_fib  # jeden program: wyjście + ślad wykonania
make cosim          # porównanie śladu z emulatorem
make fuzz N=100     # losowe programy + porównanie z emulatorem
make progs SIM=verilator
```

Rada Zrób z katalogu kursu repozytorium git i commituj po każdym zadaniu. Przy potoku (rozdział 8) możliwość powrotu do ostatniej działającej wersji jest bezcenna.

Konwencje

- SystemVerilog w podzbiorze rozumianym przez Icarus 12, Verilatora i Yosysa: `logic`, `always_comb`, `always_ff`, `typedef enum`, pakiety.
- `always_comb` z przypisaniem `=` dla logiki kombinacyjnej, `always_ff` z `<=` dla rejestrów. Bez wyjątków.
- Jeden zegar `clk`, reset *synchroniczny*, aktywny w stanie wysokim (`rst`).
- Stałe architektury zawsze z pakietu `rv32i_pkg` (`common/rtl/rv32i_pkg.sv`), nigdy „magiczne liczby”.

Rozdział 0

Narzędzia i pierwszy układ

Cel: zrozumieć, czym opis sprzętu różni się od programu, i opanować cykl *napisz* → *zasymuluj* → *obejrzyj przebiegi* → *zlintuj* → *zsyntezuj*.
Czas: 1 wieczór
Katalog: [00-start/](#)

0.1 HDL to nie jest język programowania

W C piszesz *sekwencję kroków* dla procesora, który już istnieje. W SystemVerilogu opisujesz *układ*: zbiór bramek i przerzutników połączonych przewodami. Wszystkie elementy działają *jednocześnie* i *bez przerwy*.

```
assign y = a & b; // to NIE jest "oblicz a&b i zapisz do y":  
                // to bramka AND na stałe połączona z przewodem y
```

Ten sam kod ma dwa życia:

	Symulacja	Synteza
Narzędzie	Icarus Verilog, Verilator	Yosys (dalej: nextpnr → FPGA)
Co robi	wykonuje opis jak program zdarzeniowy	zamienia opis na sieć bramek i przerzutników
Co akceptuje	prawie wszystko (#5, \$display, pliki)	tylko <i>syntezowalny podzbiór</i>

Testbench ([tb/*.sv](#)) żyje tylko w symulacji i może używać wszystkiego. Kod w [rtl/](#) musi dać się zsyntezować, co w każdej chwili sprawdzisz poleceniem `make synth T=moduł`.

Pułapka dla programisty C Kod, który się symuluje, wcale nie musi się syntezywać do tego, co myślisz. Pętla `for` to *N kopii* sprzętu, a nie *N iteracji* w czasie. `if` bez `else` w logice kombinacyjnej tworzy zatrask. Operatory `/` i `%` budują ogromny dzielnik. Zawsze pytaj: *jak wyglądałby ten układ na schemacie?*

0.2 Dwa rodzaje logiki

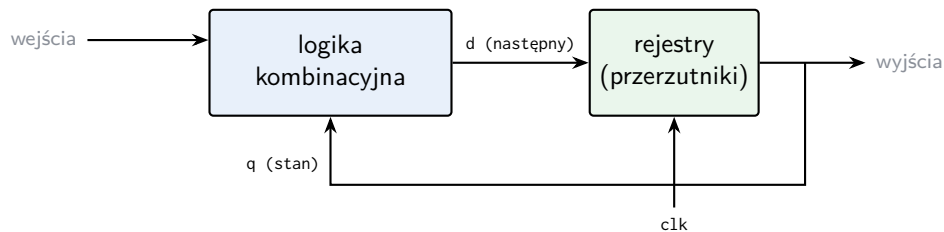
Logika kombinacyjna: wyjście zależy *tylko* od bieżących wejść, bez pamięci. Bramki, multipleksery, sumatory, dekodery.

```
always_comb y = sel ? a : b;    // multiplekser
assign sum = a + b;             // sumator
```

Logika sekwencyjna: przerzutniki (*flip-flopy*) zapamiętują wartość na zboczu zegara. To jest *stan* układu.

```
always_ff @(posedge clk) q <= d; // rejestr: na zboczu narastającym q := d
```

Prawie każdy układ synchroniczny ma tę samą budowę (rys. 0.1): logika kombinacyjna liczy *następny stan* z bieżącego stanu i wejść, a rejestry na zboczu zegara *jednocześnie* go zapamiętują.



Rysunek 0.1: Ogólny model układu synchronicznego.

0.3 Anatomia modułu

Plik `rtl/counter.sv` (gotowy przykład):

```
module counter #(
    parameter int WIDTH = 8          // parametr: stała ustalana przy tworzeniu instancji
) (
    input logic      clk,
    input logic      rst,
    input logic      en,
    output logic [WIDTH-1:0] q       // wektor WIDTH bitów, bit 0 najmłodszy
);
always_ff @(posedge clk) begin
    if (rst)    q <= '0;
    else if (en) q <= q + 1'b1;
end
endmodule
```

- `logic` to typ SystemVerilog dla sygnałów, zastępujący stare `wire` i `reg`. W kursie używamy wyłącznie `logic`.
- `[WIDTH-1:0]` to zakres bitów: `q[0]` najmłodszy, `q[WIDTH-1]` najstarszy.
- Literały mają postać `<szerokość>'<baza><wartość>`, np. `8'hFF`, `4'b1010`, `32'd100`, `1'b1`. Zapisy `'0` i `'1` oznaczają „same zera” i „same jedyńki” w dowolnej szerokości.

- Reset jest *synchroniczny* (sprawdzany na zboczu) i aktywny w stanie 1. Tej konwencji trzymamy się w całym kursie.

Instancja, czyli „wstawienie układu do innego układu”:

```
counter #(.WIDTH(16)) u_cnt (.clk(clk), .rst(rst), .en(1'b1), .q(value));
counter #(.WIDTH(16)) u_cnt (.clk, .rst, .en(1'b1), .q(value)); // .clk = .clk(clk)
```

0.4 Anatomia testbencha

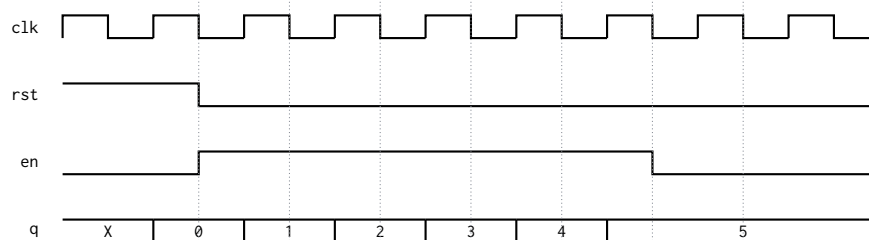
Testbench to moduł bez portów, który tworzy sygnały, wstawia testowany układ (DUT, *device under test*), generuje zegar i w bloku `initial` podaje pobudzenia oraz sprawdza wyniki:

```
module tb_counter;
  `include "tb_common.svh"           // makra CHECK_EQ, CHECK, tb_start/tb_finish
  logic clk = 0, rst, en;
  logic [7:0] q;
  counter #(.WIDTH(8)) dut (.clk, .rst, .en, .q);
  always #5 clk = ~clk;              // okres 10 jednostek czasu

  initial begin
    tb_start();
    rst = 1; en = 0;
    @(negedge clk);                  // czekaj na zbocze OPADAJĄCE
    @(negedge clk);
    `CHECK_EQ(q, 8'd0, "po resetie")
    rst = 0; en = 1;
    repeat (5) @(negedge clk);
    `CHECK_EQ(q, 8'd5, "po 5 taktach liczenia")
    tb_finish();
  end
endmodule
```

Dlaczego wejścia zmieniamy na zboczu opadającym?

Przerzutniki reagują na zbocze narastające. Gdyby testbench zmieniał wejście *dokładnie* w chwili zbocza narastającego, wynik zależałby od tego, w jakiej kolejności symulator przetworzy oba zdarzenia. To wyścig (*race*). Zmiana w połowie okresu usuwa problem (rys. 0.2).



kropki: zbocza opadające — tu testbench zmienia wejścia i sprawdza wyniki

Rysunek 0.2: Przebiegi testu licznika: **q** zmienia się tuż po zboczu narastającym, na którym **en** = 1 (a przed resetem ma wartość nieznaną **X**).

0.5 Logika czterowartościowa: 0, 1, X, Z

Symulator zna cztery wartości bitu: 0, 1, x (*nieznana*: niezainicjowany przerzutnik, konflikt dwóch sterowników albo wynik operacji na X) oraz z (nikt nie steruje przewodem; w kursie nie występuje).

X jest Twoim sprzymierzeńcem. Czerwone x w GTKWave oznacza coś niezainicjowanego albo nieprzypisanego. Prawdziwy sprzęt miałby tam losowe 0 lub 1 i błąd ujawniałby się „czasem”, a symulacja pokazuje go od razu.

Porównania z X `a == b` z X daje X, co w if działa jak *falsz*! Operatory `===` i `!==` porównują dosłownie, łącznie z X. W testbenchach używaj `===/!==` (robi to makro `CHECK_EQ`), w RTL zwykłego `==`.

0.6 Jak symulator wykonuje kod

Symulator zdarzeniowy utrzymuje kolejkę zdarzeń uporządkowaną w czasie. W jednej chwili czasu najpierw wykonuje wszystkie procesy, które się „obudziły” (zmiana sygnału z listy wrażliwości, zbocze zegara), a przypisania nieblokujące `<=` odkłada na koniec tej chwili (region NBA). Dlatego wszystkie `always_ff` widzą *stare* wartości rejestrów, tak jak prawdziwe przerzutniki taktowane jednym zegarem. Wrócimy do tego w rozdziale 2.

0.7 GTKWave w minutę

1. `make wave T=updown` otwiera GTKWave z przebiegami testu.
2. Po lewej, w drzewku *SST*, rozwiń `tb_updown`, potem `dut`.
3. Zaznacz sygnały i kliknij *Append* (albo przeciągnij je na panel).
4. `Ctrl+Shift+F`: dopasuj widok do całego czasu symulacji.
5. Prawy klik na wektorze, *Data Format: Decimal* albo *Hex*.
6. *File* → *Write Save File*: zapamiętuje układ sygnałów.

0.8 Zadania

Zadanie 0.1 — Uruchom gotowy licznik

`make unit T=counter`, potem `make wave T=counter`. Dodaj `clk`, `rst`, `en`, `q`. Znajdź moment, w którym `en` spada do 0, i sprawdź, czy `q` się zatrzymuje. W którym dokładnie momencie zmienia się `q` i względem którego zbocza?

Zadanie 0.2 — Licznik w górę/w dół (`rtl/updown.sv`)

Priorytety: `rst` > `load` > `en`. Przy `en = 1` licznik liczy w górę albo w dół (`up`) z zawijaniem. Wyjście kombinacyjne `tc` (*terminal count*) = 1 dokładnie wtedy, gdy `en = 1` i następny krok spowoduje zawinięcie. Test porównuje układ z modelem przez 500 losowych taktów i wypisuje

pierwsze rozbieżności z czasem symulacji. Użyj go, żeby znaleźć to miejsce w GTKWave.

Zadanie 0.3 — Pierwsza synteza

`make synth T=counter`. Yosys wypisze m.in. 8 komórek `$_SDFE_PP0P_`: przerzutnik z synchronicznym resetem do 0 i zezwoleniem, czyli dokładnie to, co opisałeś. Zmień domyślne `WIDTH` na 32 i porównaj liczbę bramek oraz najdłuższą ścieżkę. Dlaczego ścieżka rośnie? (Podpowiedź: przeniesienie w dodawaniu.)

Zadanie 0.4 — Lint

`make lint` powinno zgłosić OK. Dla eksperymentu przypisz `q <= d[1:0]`; przy `WIDTH = 4` i zobacz ostrzeżenie o szerokości. Verilator to Twój `-Wall -Werror`.

Moduł zaliczony, gdy...

`make test` pokazuje PASS dla counter i updown, a `make lint` nie zgłasza ostrzeżeń.

Pytania kontrolne

1. Czym różni się `assign y = a + b;` od `y = a + b;` w funkcji C?
2. Dlaczego testbench zmienia wejścia na zboczu opadającym zegara?
3. Co oznacza X w przebiegach i czemu `if (x == 1)` przy `x = X` wybiera gałąź `else`?
4. Które konstrukcje z `tb_counter.sv` nie dałyby się zsyntezować i dlaczego to nie problem?

Rozdział 1

Logika kombinacyjna

Cel: pisać logikę kombinacyjną świadomie: wiedzieć, jaki sprzęt powstaje z każdej linii, i nie dać się złapać na szerokości wyrażen ani zatrzaski.
Czas: 2 wieczory
Katalog: 01-kombinacyjna/

1.1 assign i always_comb

Obie konstrukcje opisują logikę kombinacyjną:

```
assign y = sel ? a : b;

always_comb begin
    if (sel) y = a;
    else    y = b;
end
```

W `always_comb` używamy przypisania *blokującego* =. Wewnątrz bloku działa jak w C, bo kolejne linie widzą wyniki poprzednich, ale cały blok to *jedna* sieć bramek liczona „natychmiast”. Zmienna pośrednia jest tylko nazwą dla przewodu:

```
always_comb begin
    t = a ^ b;          // t: nazwa dla przewodu z wyjścia bramki XOR
    y = t & c;
end
```

`always_comb` (w odróżnieniu od starego `always @(*)`) informuje też narzędzia o Twoim zamiarze: jeśli wyjdzie z niego coś, co nie jest kombinacyjne (zatrzask), Yosys odmówi syntezy.

1.2 Wektory: wycinanie, sklejanie, powielanie

```
logic [31:0] x;
x[7:0]          // najmłodszy bajt
x[31]           // bit znaku
x[8*i+: 8]      // bajt i: 8 bitów od pozycji 8*i w górę (i może być zmienną)
{a, b}          // sklejenie: a na starszych bitach
{4{b}}          // b powtórzone 4 razy
```

```
{{20{x[31]}}, x[31:20]} // rozszerzenie znakiem 12 -> 32 bity (rozdział 4!)
{cout, sum} = a + b;      // sklejanie po lewej: wynik z przeniesieniem
```

1.3 Liczby ze znakiem w sprzęcie

Sprzęt nie „wie”, czy wektor bitów jest liczbą ze znakiem. Kod uzupełnień do dwóch (U2) jest wybrany właśnie dlatego, że *ten sam sumator* poprawnie dodaje liczby ze znakiem i bez znaku. Znak ma znaczenie tylko w kilku operacjach:

Operacja	bez znaku	ze znakiem
porównanie <	$a < b$	$\$signed(a) < \$signed(b)$
przesunięcie w prawo	$a \gg n$ (wsuwa 0)	$\$signed(a) \gg n$ (powiela bit znaku)
rozszerzanie do szerszego typu	zerami	kopiami bitu znaku
mnożenie (górna połowa), dzielenie	osobne układy	osobne układy

Negacja w U2: $-x = \bar{x} + 1$. Stąd odejmowanie $a - b = a + \bar{b} + 1$, czyli *sumator z negacją jednego wejścia i przeniesieniem wejściowym 1*. Z tego triku skorzysta ALU w rozdziale 3.

1.4 Szerokości wyrażeń: najczęstsze źródło błędów

W C typ wyrażenia zależy od typów argumentów. W SystemVerilogu **szerokość obliczeń zależy też od lewej strony przypisania**:

```
logic [7:0] a, b, s8;
logic [8:0] s9;
s8 = a + b; // liczone na 8 bitach: przeniesienie ginie
s9 = a + b; // liczone na 9 bitach: przeniesienie trafia do s9[8]
```

- Wyrażenie arytmetyczne liczy się w szerokości $\max(\text{operandy, lewa strona})$. Węższe operandy są rozszerzane.
- Liczba bez rozmiaru (5) ma 32 bity i jest *signed*. 1'b1 ma 1 bit.
- Jeśli którykolwiek operand jest **unsigned**, całe wyrażenie jest **unsigned**. `logic` jest domyślnie **unsigned**.

»> bez \$signed `a >> n` na wektorze **unsigned** to zwykle `>>`. Przesunięcie arytmetyczne wymaga `$signed(a) >> n`. Podobnie `a < b` na `logic` porównuje bez znaku. Ten błąd wychodzi dopiero na liczbach ujemnych, dlatego testy kursu celowo używają `0x80000000` i `0xFFFFFFFF`.

Verilator (`make lint`) ostrzega o niejawnym obcinaniu i rozszerzaniu (`WIDTHTRUNC`, `WIDTHEXPAND`). Traktuj te ostrzeżenia poważnie.

1.5 Jaki sprzęt powstaje

Konstrukcja	Sprzęt	Koszt
$\& \mid \wedge \sim$	bramki, po jednej na bit	1 poziom
$c ? a : b, \text{if/else}$	multiplekser 2:1	tani
case na N wartości	multiplekser $N:1$ (drzewo, $\log_2 N$ poziomów)	umiarkowany
$a == b$	XOR-y i drzewo OR	tani
$a + b, a - b, a < b$	sumator (łańcuch przeniesień)	średni
$a \ll b$ (b zmienne)	$\log_2 N$ warstw multiplekserów	spory
$a * b$	macierz sumatorów	drogi
$a / b, a \% b$	sieć odejmowań	bardzo drogi

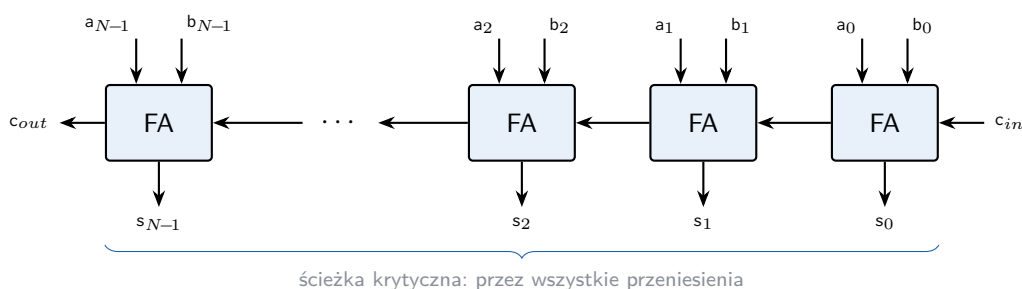
Ścieżka krytyczna. Sygnał musi przejść przez najdłuższy łańcuch bramek w ciągu jednego taktu zegara. Im dłuższy łańcuch, tym niższa maksymalna częstotliwość. `make synth` podaje to jako *longest topological path*, czyli liczbę bramek na najdłuższej drodze. To przybliżenie, bo prawdziwe opóźnienia zależą od technologii, ale dobrze pokazuje trendy.

1.6 Przykład: sumator ripple-carry

Sumator pełny (*full adder*) dodaje trzy bity:

$$s = a \oplus b \oplus c_{in}, \quad c_{out} = ab + c_{in}(a \oplus b).$$

Łącząc N takich sumatorów łańcuchem przeniesień, dostajemy sumator N -bitowy (rys. 1.1). Przeniesienie z bitu 0 może wpłynąć na bit $N - 1$, więc ścieżka krytyczna jest proporcjonalna do N : $O(n)$.



Rysunek 1.1: Sumator ripple-carry z N sumatorów pełnych.

Szybsze sumatory (carry-lookahead, Kogge–Stone) liczą przeniesienia równolegle w $O(\log n)$ poziomach kosztem większej liczby bramek. Na FPGA operator `+` trafia na dedykowane łańcuchy przeniesień, wielokrotnie szybsze od sumatora złożonego ręcznie z LUT-ów. Wniosek praktyczny: **pisz `+` i pozwól narzędziom wybrać implementację**. Ręczna budowa ma sens tylko w celach dydaktycznych albo gdy wiesz, po co.

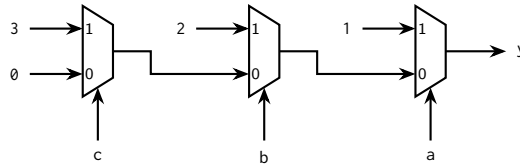
1.7 if kontra case i priorytety

```

always_comb begin           // łańcuch priorytetów: a wygrywa z b, b z c
    if (a)      y = 1;
    else if (b) y = 2;
    else if (c) y = 3;
    else       y = 0;
end

```

Kolejne else if to kaskada multiplekserów (rys. 1.2). Warunek z *pierwszego* if jest najbliżej wyjścia, więc ma najwyższy priorytet. Gdy warunki się wykluczają (np. różne wartości jednego sygnału), case lepiej oddaje intencję i daje synteze swobodę zbudowania płaskiego drzewa.



Rysunek 1.2: if / else if / else jako łańcuch multiplekserów 2:1. Wejście 1 jest wybierane, gdy warunek (nóżka u dołu) jest prawdziwy; warunek a jest najbliżej wyjścia, więc ma najwyższy priorytet.

1.8 Zatrzaski: błąd numer jeden

```

always_comb begin
    if (en) y = a;    // a gdy en == 0? y ma "pamiętać" starą wartość!
end

```

Skoro y ma pamiętać wartość przy $en = 0$, syntezy musi wstawić element pamiętający, czyli *zatrzask* sterowany poziomem. W układach synchronicznych to prawie zawsze błąd: psuje analizę czasową, a układ zachowuje się inaczej na krzemie niż w symulacji.

Lekarstwo: każda zmienna przypisywana w `always_comb` musi dostać wartość na *każdej* ścieżce. Najprościej przez przypisanie domyślne na początku bloku:

```

always_comb begin
    y = '0;           // wartość domyślna
    if (en) y = a;
end

```

To samo dotyczy case bez default, gdy nie wszystkie wartości są pokryte.

1.9 Pętle for i generate: kopiowanie sprzętu

```

always_comb begin
    parity = 1'b0;
    for (int i = 0; i < 32; i++)
        parity = parity ^ x[i];    // 32 bramki XOR połączone łańcuchem
end

```

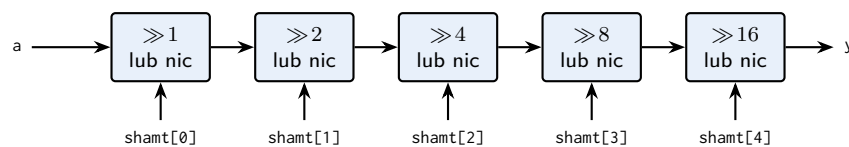
Pętla jest **rozwijana w czasie syntezy**: nie ma tu licznika ani iteracji w czasie, tylko 32 kopie logiki. Liczba iteracji musi być stała. (Syntezator zwykle przebuduje ten łańcuch w drzewo o głębokości $\log_2 32$, bo XOR jest łączny.)

Do powielania *instancji* modułów służy `generate`:

```
genvar i;
generate
  for (i = 0; i < WIDTH; i++) begin : g_fa      // etykieta bloku jest wymagana
    full_adder fa (.a(a[i]), .b(b[i]), .cin(c[i]), .s(sum[i]), .cout(c[i+1]));
  end
endgenerate
```

1.10 Przykład: przesuwnik logarytmiczny

Przesunięcie o zmienną liczbę bitów 0..31 można zbudować z 5 warstw multiplekserów: warstwa k przesuwą o 2^k albo nie, zależnie od bitu `shamt[k]` (rys. 1.3). Każda liczba 0..31 jest sumą niektórych potęg dwójki. Przesunięcie w lewo uzyskuje się, odwracając kolejność bitów, przesuwając w prawo i odwracając z powrotem.



Rysunek 1.3: Barrel shifter: 5 warstw po 32 multipleksery 2:1.

1.11 Znane ograniczenia Icarus Verilog 12

Icarus nie obsługuje całego SystemVeriloga. W RTL rzadko to przeszkadza, ale warto wiedzieć:

- brak `break` i `continue` w pętlach,
- brak inicjalizacji całej tablicy w deklaracji (`logic [7:0] t [4] = '{...}`),
- `unique case` i `priority case` są akceptowane, ale ignorowane,
- komunikat *sorry: constant selects in always_* processes* to tylko ostrzeżenie (Makefile go ukrywa).

1.12 Zadania

Zadanie 1.1 — Multiplekser (rtl/mux4.sv)

Dwie wersje: `always_comb` z `case` oraz jeden `assign` z operatorem `?:`. Test sprawdza instancje 8- i 16-bitową, czyli też to, czy parametr działa.

Zadanie 1.2 — Enkoder priorytetowy (rtl/prio_enc.sv)

idx = numer najstarszego ustawionego bitu, valid = czy jakikolwiek. Użyj pętli for. Pętla od bitu 0 do 7, w której każde trafienie nadpisuje idx, daje regułę „najstarszy wygrywa”. Dlaczego? Narysuj powstały łańcuch multiplekserów.

Zadanie 1.3 — Sumator ripple-carry (rtl/adder.sv)

Najpierw full_adder z samych bramek, potem adder przez generate, bez operatora +. Test sprawdza *wszystkie* kombinacje dla 8 bitów (2^{17} przypadków) i losowe dla 32. Potem make synth T=adderr: zanotuj liczbę komórek i ścieżkę, zastąp ciało modułu wersją assign {cout, sum} = a + b + cin; i porównaj.

Zadanie 1.4 — Barrel shifter (rtl/shifter.sv)

SLL, SRL i SRA o shift z 5 warstw multiplekserów, bez operatorów przesunięcia. Ile multiplekserów 2:1 ma Twój układ (bez odwracania)? Porównaj z make synth T=shifter.

Zadanie 1.5 — Polowanie na zatrask (rtl/hex7seg.sv)

Dekoder 7-segmentowy ma dwa błędy prowadzące do zatrasku. Zanim cokolwiek poprawisz, uruchom make lint, make synth T=hex7seg i make unit T=hex7seg. Zauważ, że lint znajduje tylko jeden z dwóch błędów, a Yosys odmawia syntezy z komunikatem *Latch inferred*. Potem popraw kod i dokończ tablicę A–F.

Moduł zaliczony, gdy...

make test → 5 × PASS, make lint → wszystko OK.

Pytania kontrolne

1. Ile bitów ma wynik $a + b$ dla 8-bitowych a, b przy przypisaniu do 16-bitowej zmiennej? A do 8-bitowej?
2. Co da $\$signed(8'hF0) \gg 2$, a co $8'hF0 \gg 2$?
3. Dlaczego if bez else w always_comb tworzy zatrask, a w always_ff nie?
4. Czym różni się pętla for w RTL od pętli w C? Co jest stałe w czasie syntezy?
5. Jak głęboka (w poziomach logiki) jest ścieżka krytyczna sumatora ripple-carry dla N bitów, a jak przesuwnika logarytmicznego?

Rozdział 2

Logika sekwencyjna i automaty stanów

Cel: rejestry, czas w układzie synchronicznym, pamięci i automaty skończone. Procesor to w gruncie rzeczy wielki automat: jego stanem są PC, rejestry i pamięć.
Czas: 2–3 wieczory
Katalog: [02-sekwencyjna/](#)

2.1 Przerzutnik i always_ff

```
always_ff @(posedge clk) q <= d;
```

Na każdym zboczu narastającym clk przerzutnik zapamiętuje d. Między zboczami q się nie zmienia, cokolwiek dzieje się z d.

Dlaczego przypisanie nieblokujące <=?

<= oznacza: *oblicz prawą stronę teraz, przypisz na końcu kroku czasowego*. Wszystkie <= w całym układzie widzą więc *stare* wartości, dokładnie tak jak prawdziwe przerzutniki taktowane jednym zegarem:

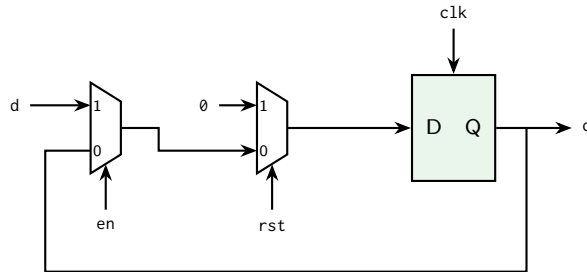
```
always_ff @(posedge clk) begin // rejestr przesuwany 3-bitowy
    a <= in;
    b <= a; // stare a
    c <= b; // stare b
end

always_ff @(posedge clk) begin // zamiana wartości: działa!
    x <= y;
    y <= x;
end
```

= w always_ff Z = linia b = a widziałaby *nowe* a i rejestr przesuwany zapadłby się w jeden przerzutnik. Przy kilku blokach always wynik zależałby od kolejności ich wykonania przez symulator. Zasada bez wyjątków: **w always_ff tylko <=**.

Rejestr z zezwoleniem i resetem

```
always_ff @(posedge clk) begin
  if (rst)    q <= '0;      // reset synchroniczny: działa tylko na zboczu
  else if (en) q <= d;      // en = 0: q <= q
end
```



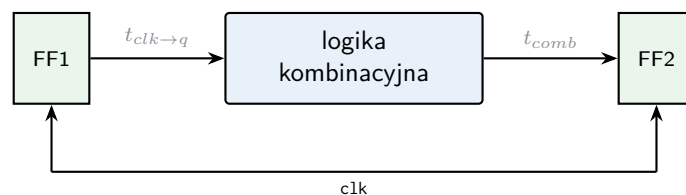
Rysunek 2.1: Rejestr z resetem synchronicznym i zezwoleniem: dwa multipleksery i przerzutnik D.

Sprzętowo (rys. 2.1) to przerzutnik D z dwoma multipleksarami przed nim. Biblioteki FPGA i ASIC mają gotowe przerzutniki z wejściem *enable* (\$_SDFFE_ z rozdziału 0).

Reset synchroniczny czy asynchroniczny? Asynchroniczny (always_ff @(posedge clk or posedge rst)) działa bez zegara, ale komplikuje analizę czasową przy zdejmowaniu resetu. W kursie używamy tylko synchronicznego, który na FPGA jest standardem.

Wartości początkowe. Na FPGA przerzutniki dostają wartości z bitstreamu, więc initial i logic $q = 0$; są syntezywalne. W ASIC po włączeniu zasilania stan jest losowy i tylko reset daje pewność. Dlatego wszystko, co musi mieć znaną wartość (PC, stan automatu, bity valid), resetujemy jawnie.

2.2 Czas w układzie synchronicznym



Rysunek 2.2: Ścieżka między dwoma rejestrami.

Żeby FF2 poprawnie zapamiętał wynik, sygnał musi do niego dotrzeć przed zboczem z zapasem t_{setup} :

$$T_{clk} \geq t_{clk \rightarrow q} + t_{comb}^{max} + t_{setup}.$$

Najdłuższa ścieżka kombinacyjna między dwoma rejestrami (**ścieżka krytyczna**) wyznacza maksymalną częstotliwość zegara. Tak wygląda „złożoność obliczeniowa” sprzętu: nie liczba operacji, tylko *głębokość logiki między rejestrami*. Za długą ścieżkę można przeciąć rejestrem. Na tym polega **potokowanie** (rozdział 8).

W symulacji RTL czasów nie ma, bo logika „liczy się w zerowym czasie”. O timing dbają synteza i place&route. Przybliżenie pokazuje `make synth (longest topological path)`.

2.3 Pamięci

```
logic [31:0] mem [0:255]; // 256 słów po 32 bity
always_ff @(posedge clk)
  if (we) mem[waddr] <= wdata; // zapis synchroniczny
assign rdata = mem[raddr]; // odczyt asynchroniczny
// albo:
always_ff @(posedge clk) rdata <= mem[raddr]; // odczyt synchroniczny
```

Odczyt synchroniczny (dana dostępna takt później): tak działają bloki RAM w FPGA (BRAM) i pamięci SRAM. Są gęste i tanie.

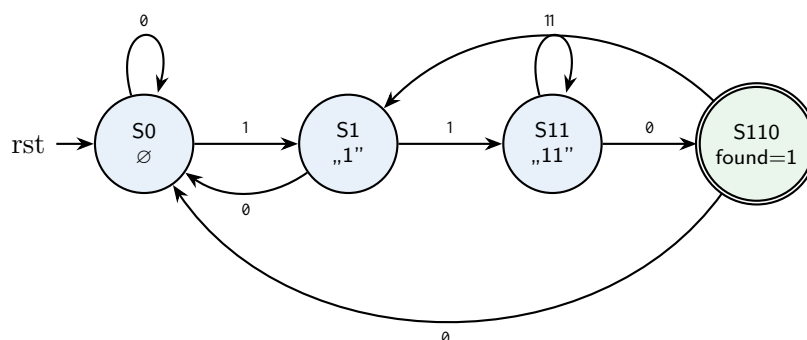
Odczyt asynchroniczny wymaga zbudowania pamięci z przerzutników i multiplexerów albo z „rozproszonej” pamięci LUT. Dla dużych pamięci jest drogi.

Pamięć w testbenchu naszego procesora ma odczyt asynchroniczny, bo upraszcza to procesor jednocyklowy. Bank rejestrów (32×32 bity) jest mały i też ma odczyt asynchroniczny. W rozdziale 3 zobaczysz, ile to kosztuje.

2.4 Automaty skończone (FSM)

Automat to rejestr stanu i dwie funkcje kombinacyjne: *następny stan* = $f(\text{stan}, \text{wejścia})$ oraz *wyjścia* = $g(\text{stan})$ w automacie **Moore’a** albo $g(\text{stan}, \text{wejścia})$ w automacie **Mealy’ego**. Wyjścia Moore’a zmieniają się tylko po zboczu zegara (są „czyste”, ale opóźnione o takt). Mealy reaguje w tym samym takcie, ale jego wyjście zależy kombinacyjnie od wejść (dłuższe ścieżki, ryzyko pętli kombinacyjnych między modułami).

Przykład: detektor sekwencji 110 (Moore, z nakładaniem)



Rysunek 2.3: Automat Moore’a wykrywający 110. Wyjście found zależy tylko od stanu.

Każda strzałka na diagramie to jedna gałąź w case. Styl obowiązujący w kursie, czyli **dwa procesy**:

```

typedef enum logic [1:0] {S0, S1, S11, S110} state_t;
state_t state, next;

always_ff @(posedge clk)           // 1) tylko rejestr stanu
    if (rst) state <= S0;
    else     state <= next;

always_comb begin                  // 2) tylko logika następnego stanu
    next = state;                  // wartość domyślna (brak zatrzasku!)
    case (state)
        S0:  if (in) next = S1;   else next = S0;
        S1:  if (in) next = S11;  else next = S0;
        S11: if (in) next = S11;  else next = S110;
        S110: if (in) next = S1;   else next = S0;
        default: next = S0;
    endcase
end

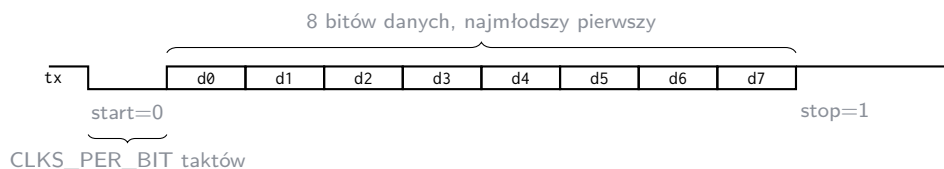
assign found = (state == S110);    // wyjście Moore'a

```

Icarus i enum next = in ? S1 : S0; daje w Icarusie błąd *This assignment requires an explicit cast*. Pisz if/else (jak wyżej) albo next = state_t'(in ? S1 : S0);.

Automat z licznikiem: sterowanie i ścieżka danych

Większość prawdziwych układów (UART, kontroler pamięci, procesor wielocyklowy) to mały automat sterujący i „ścieżka danych”: liczniki i rejestry przesuwne. Automat mówi „licz”, „przesuń”, „załaduj”, a ścieżka danych odpowiada „doliczyłem do końca”. Nadajnik UART z zadania 2.5 jest dokładnie taki.



Rysunek 2.4: Ramka UART 8N1. Linia w spoczynku ma stan 1.

2.5 Porównania z parametrami

parameter int DEPTH = 8 ma 32 bity. Porównanie count == DEPTH, gdzie count ma 4 bity, jest poprawne, ale Verilator zgłosi WIDTHEXPAND. Czysty idiom to stała o właściwej szerokości:

```

localparam int AW = $clog2(DEPTH);
localparam logic [AW:0] FULL = (AW+1)'(DEPTH);    // rzutowanie na AW+1 bitów
assign full = (count == FULL);

```

2.6 Kolejka FIFO: wskaźniki z dodatkowym bitem

FIFO o głębokości $D = 2^k$ to pamięć i dwa wskaźniki: zapisu i odczytu. Gdyby wskaźniki miały k bitów, stan „pełna” i „pusta” wyglądałyby tak samo (wskaźniki równe). Sztuczka: wskaźniki mają $k + 1$ bitów, a pamięć adresuje się młodszymi k bitami. Wtedy:

- `count = wr_ptr - rd_ptr`: odejmowanie modulo 2^{k+1} zawija się samo,
- `empty`: wskaźniki równe,
- `full`: różnią się tylko najstarszym bitem (czyli `count == D`).

2.7 Sygnały z zewnątrz (na przyszłość)

Przycisk albo linia RX z UART zmieniają się niezależnie od Twojego zegara. Przerzutnik próbujący taki sygnał może wpaść w **metastabilność**. Standardowe lekarstwo to dwa przerzutniki pod rząd (synchronizator) przed jakąkolwiek logiką. W symulacji tego nie zobaczysz, a na płytce objawia się „losowymi” błędami raz na godzinę.

2.8 Zadania

Zadanie 2.1 — Detektor zbocza (`rtl/edge_detect.sv`)

Jeden przerzutnik i jedna bramka. W przebiegach (`make wave T=edge_detect`) zobacz, że `rise` jest kombinacyjny: pojawia się razem ze zmianą `in`, a znika na zboczu zegara.

Zadanie 2.2 — LFSR (`rtl/lfsr16.sv`)

Wielomian $x^{16} + x^{14} + x^{13} + x^{11} + 1$, ziarno 16'hACE1. Test sprawdza też okres (65535). Dlaczego okres to $2^{16} - 1$, a nie 2^{16} ? Jakiego stanu LFSR nigdy nie osiągnie i co by się stało, gdyby w nim wystartował?

Zadanie 2.3 — Detektor 1011 (`rtl/seq_detect.sv`)

Automat Moore’a, 5 stanów, styl dwuprocesowy. **Najpierw narysuj diagram** jak na rys. 2.3. Najtrudniejsze są przejścia przy nakładaniu: dokąd przejść ze stanu „1011”, gdy przyjdzie 0, a dokąd, gdy 1?

Zadanie 2.4 — FIFO (`rtl/fifo.sv`)

Test porównuje FIFO z kolejką `q[$]` z SystemVeriloga przez 3000 losowych taktów, także przy jednoczesnym `push` i `pop`.

Zadanie 2.5 — Nadajnik UART (`rtl/uart_tx.sv`)

Ramka z rys. 2.4. Test to „odbiornik programowy” w testbenchu (warto przeczytać: `fork/join`, taski czekające na zbocza). Po zrobieniu: `make synth T=uart_tx`. Czy umiesz przypisać każdy przerzutnik do konkretnego rejestru w swoim kodzie? Ten moduł przyda się na FPGA (rozdział 9).

Moduł zaliczony, gdy...

make test → 5 × PASS, make lint czysto.

Pytania kontrolne

1. Co dokładnie zrobi symulator z $a \leq b$; $b \leq a$; na zboczu zegara? A z $a = b$; $b = a$;
2. Od czego zależy maksymalna częstotliwość zegara układu? Co to jest ścieżka krytyczna?
3. Czym różni się odczyt synchroniczny od asynchronicznego i który z nich mają bloki RAM w FPGA?
4. Narysuj automat Mealy'ego wykrywający 110. Ile ma stanów? Kiedy zmienia się jego wyjście?
5. Dlaczego wskaźniki FIFO mają o jeden bit więcej, niż potrzeba do adresowania?

Rozdział 3

ALU, bank rejestrów, komparator skoków

Cel: zbudować trzy największe bloki ścieżki danych RV32I. Od tego rozdziału interfejsy modułów są kontraktem: procesory z rozdziałów 5 i 8 użyją dokładnie tych portów.

Czas: 1–2 wieczory

Katalog: [03-alu-regfile/](#)

3.1 RV32I w pigułce

RISC-V RV32I to 32-bitowa architektura typu *load/store*:

- **32 rejestry** $x0$ – $x31$ po 32 bity, przy czym **$x0$ jest zawsze zerem**, a zapis do niego jest ignorowany. Dzięki temu wiele pseudoinstrukcji nie kosztuje: `mv a,b = addi a,b,0`, `nop = addi x0,x0,0`, `j L = jal x0,L`.
- Arytmetyka działa tylko na rejestrach, a do pamięci dostęp jest wyłącznie przez `load` i `store`.
- **Brak flag:** nie ma rejestru statusu jak w x86 czy ARM. Skoki warunkowe same porównują dwa rejestry (`blt a0, a1, L`), dlatego zamiast flag z ALU budujemy osobny komparator.

Rejestr	Nazwa ABI	Rola
$x0$	zero	stałe zero
$x1$	ra	adres powrotu
$x2$	sp	wskaźnik stosu
$x3$	gp	global pointer (w testach kursu: numer podtestu)
$x5$ – 7 , $x28$ – 31	t0–t6	tymczasowe (<i>caller-saved</i>)
$x8$ – 9 , $x18$ – 27	s0–s11	zachowywane (<i>callee-saved</i>), s0 = fp
$x10$ – 17	a0–a7	argumenty i wynik funkcji

3.2 ALU

Dziesięć operacji wystarcza na całą arytmetykę RV32I:

Kod op	Operacja	Używana przez
ALU_ADD 0_000	$a + b$	add, addi, adresy load/store, auipc, lui ($0 + imm$), jalr
ALU_SUB 1_000	$a - b$	sub
ALU_SLL 0_001	$a \ll b[4:0]$	sll, slli
ALU_SLT 0_010	$a <_s b ? 1 : 0$	slt, slti
ALU_SLTU 0_011	$a <_u b ? 1 : 0$	sltu, sltiu
ALU_XOR 0_100	$a \oplus b$	xor, xori
ALU_SRL 0_101	$a \gg b[4:0]$ (zera)	srl, srli
ALU_SRA 1_101	$a \gg b[4:0]$ (znak)	sra, srai
ALU_OR 0_110	$a \vee b$	or, ori
ALU_AND 0_111	$a \wedge b$	and, andi

Kody są celowo równe {funct7[5], funct3} instrukcji R-type. Dekoder z rozdziału 5 dla add/sub/sll/... po prostu przepisze te bity z instrukcji. Stałe pochodzą z pakietu, który importujesz w nagłówku modułu:

```
module alu import rv32i_pkg::*; (
  input logic [31:0] a, b,
  input logic [3:0] op,
  output logic [31:0] y
);
always_comb begin
  case (op)
    ALU_ADD: y = a + b;
    // ...
```

Trzy klasyczne błędy w ALU (1) $a \gg n$ bez $\$signed(a)$ to zwykłe przesunięcie logiczne. (2) $a < b$ na logic porównuje bez znaku, więc SLT wymaga $\$signed$. (3) Przesuwa się o $b[4:0]$, nie o całe b : ISA definiuje sll x1,x2,x3 jako przesunięcie o $x3 \& 31$.

Współdzielony sumator (dla chętnych)

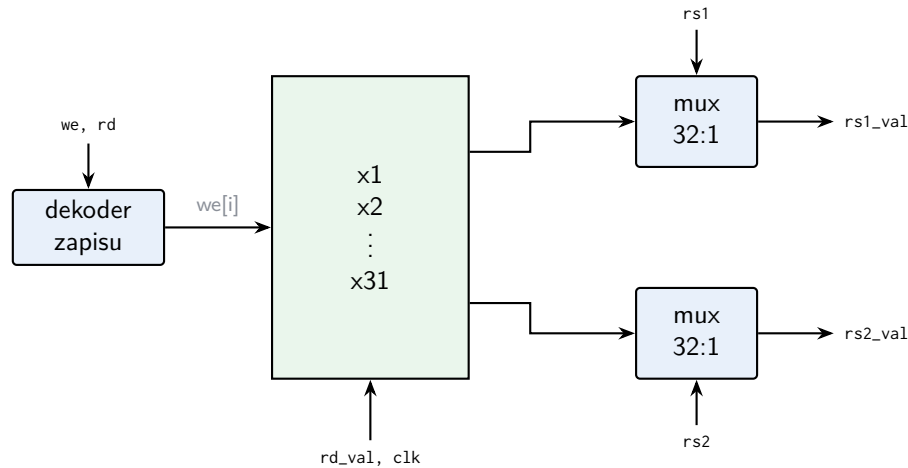
Prosta wersja z case buduje osobny sumator dla ADD, osobny dla SUB i jeszcze jeden dla SLT/SLTU. Wszystkie da się obsłużyć *jednym*: $a - b = a + \bar{b} + 1$, SLTU to brak przeniesienia z odejmowania, a SLT to znak różnicy poprawiony o przepełnienie:

$$SLT = (a_{31} \neq b_{31}) ? a_{31} : (a - b)_{31}.$$

Porównaj make synth T=alu przed i po. Yosys część takiego współdzielenia robi sam, części nie.

3.3 Bank rejestrów

- **x0**: najprościej trzymać tablicę regs[1:31], przy odczycie dla $rs == 0$ zwracać 0, a zapis z $rd == 0$ ignorować.
- **Odczyt w takcie zapisu** pod ten sam adres zwraca *starą* wartość. W procesorze jednocyklowym to poprawne: instrukcja czyta rejestry na początku taktu, a wynik zapisuje na jego końcu. W potoku (rozdział 8) „bypass” dodasz obok banku, nie wewnątrz.



Rysunek 3.1: Bank rejestrów: 31 rejestrów (x_0 jest stałą), dwa multipleksery odczytu 32:1 i dekodery zapisu.

- **Zerowanie w initial.** Prawdziwy procesor ma po resecie śmieci w rejestrach. U nas zerujemy je z powodu praktycznego: w rozdziale 6 porównujemy ślad procesora z emulatorem, który startuje od zer. Kompilatory C zapisują na stos niezainicjowane rejestry s^* w prologach funkcji, więc RTL miałby tam x , emulator 0, i porównanie by się rozjechało. Na FPGA initial jest synteżowalne, w ASIC nie.

Ile to kosztuje?

Synteza wzorcowego rozwiązania daje 992 przerzutniki (31×32) i ok. 1900 multiplekserów 2:1. Każdy port odczytu to 32 drzewa po 31 multiplekserów. Bank rejestrów jest *dwa razy większy niż cała ALU*! Dlatego prawdziwe procesory na FPGA często trzymają rejestry w BRAM-ie z odczytem synchronicznym, a odczyt rejestrów jest wtedy osobnym etapem potoku.

3.4 Komparator skoków

funct3	Instrukcja	Warunek skoku	Uwagi
000	beq	$a = b$	
001	bne	$a \neq b$	negacja beq
100	blt	$a <_s b$	ze znakiem
101	bge	$a \geq_s b$	negacja blt
110	bltu	$a <_u b$	bez znaku
111	bgeu	$a \geq_u b$	negacja bltu

Struktura kodowania: funct3[0] neguje wynik, a funct3[2:1] wybiera rodzaj porównania. Da się to zapisać krócej niż przez sześć przypadków case.

3.5 Zadania

Zadanie 3.1 — ALU (`rtl/alu.sv`)

Test: 20 000 kombinacji z naciskiem na przypadki brzegowe (0, 1, -1, `0x80000000`, `0x7FFFFFFF`, przesunięcie o 33).

Zadanie 3.2 — Bank rejestrów (`rtl/regfile.sv`)

Test sprawdza zerowy stan początkowy, x0, brak zapisu przy `we = 0` i *stary* odczyt w trakcie zapisu.

Zadanie 3.3 — Komparator (`rtl/branch_cmp.sv`)

Zadanie 3.4 — Synteza i koszt

`make synth T=alu, T=regfile, T=branch_cmp`. Zapisz liczby komórek i długości ścieżek. Który blok jest największy i dlaczego? Która operacja ALU wyznacza najdłuższą ścieżkę? (Zakomentuj na chwilę ADD/SUB/SLT i zsyntezuj ponownie.)

Moduł zaliczony, gdy...

`make test` → 3 × PASS, `make lint` czysto.

Pytania kontrolne

1. Dlaczego RISC-V nie ma flag i jakie to ma konsekwencje dla sprzętu?
2. Jaki wynik da SLT dla $a = 0x80000000$, $b = 1$? A SLTU?
3. Dlaczego odczyt w trakcie zapisu ma zwracać starą wartość w procesorze jednocyklowym?
4. Ile multiplekserów 2:1 potrzeba na jeden 32-bitowy port odczytu z 32 rejestrów?

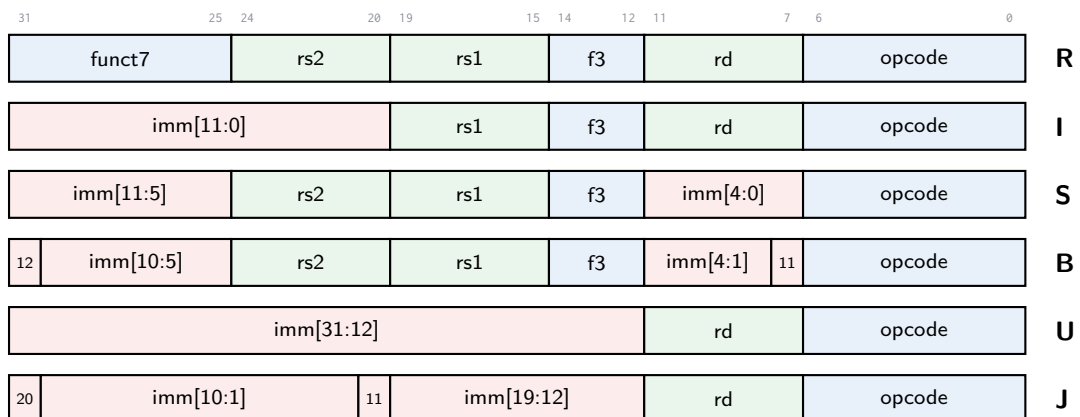
Rozdział 4

Architektura RV32I i emulator w C

Cel: znać RV32I na poziomie bitów i napisać *model referencyjny* procesora w C: emulator, z którym w rozdziałach 6–8 porównasz swój sprzęt instrukcja po instrukcji.
Czas: 2–3 wieczory
Katalog: [04-isa-emulator/](#)

4.1 Formaty instrukcji

Każda instrukcja ma 32 bity. **Pola rejestrów są zawsze w tych samych miejscach**, więc dekodery może czytać rejestry, zanim jeszcze rozpozna instrukcję.



Rysunek 4.1: Sześć formatów instrukcji RV32I (f3 = funct3). Na czerwono fragmenty stałej natychmiastowej.

4.2 Stałe natychmiastowe (immediate)

Format	Immediate jako 32-bitowa liczba (i = instrukcja)
I	$\{\{20\{i[31]\}\}, i[31:20]\}$
S	$\{\{20\{i[31]\}\}, i[31:25], i[11:7]\}$
B	$\{\{19\{i[31]\}\}, i[31], i[7], i[30:25], i[11:8], 1'b0\}$
U	$\{i[31:12], 12'b0\}$
J	$\{\{11\{i[31]\}\}, i[31], i[19:12], i[20], i[30:21], 1'b0\}$

Dlaczego bity są tak „pomieszczone”? Zasada projektowa RISC-V: każdy bit wyniku ma pochodzić z jak najmniejszej liczby pozycji w instrukcji.

- Bit znaku to *zawsze* `instr[31]`, więc rozszerzanie znakiem zaczyna się, zanim dekodery rozpozna format.
- Bity `imm[10:5]` są zawsze w `instr[30:25]`, a `imm[4:1]` w jednym z dwóch miejsc.
- W sprzęcie każdy bit immediate’a to mały multiplexer o 2–3 wejściach zamiast dużego.

W formatach B i J nie ma bitu 0, bo skoki są zawsze o parzystą liczbę bajtów (rozszerzenie C ma instrukcje 16-bitowe). Dzięki temu zasięg jest dwa razy większy: ± 4 KiB dla B i ± 1 MiB dla J.

4.3 Lista instrukcji RV32I

opcode	Format	Instrukcje	Semantyka
01101111 LUI	U	lui rd, imm	$rd = imm \ll 12$
00101111 AUIPC	U	auipc rd, imm	$rd = pc + (imm \ll 12)$
11011111 JAL	J	jal rd, off	$rd = pc + 4; pc += off$
11001111 JALR	I	jalr rd, off(rs1)	$t = (rs1 + off) \& \sim 1; rd = pc + 4; pc = t$
11000111 BRANCH	B	beq bne blt bge bltu bgeu	jeśli warunek(rs1, rs2): $pc += off$
00000111 LOAD	I	lb lh lw lbu lhu	$rd = \text{rozszerz}(\text{mem}[rs1 + off])$
01000111 STORE	S	sb sh sw	$\text{mem}[rs1 + off] = rs2$ (8/16/32 b)
00100111 OP-IMM	I	addi slti sltiu xori ori andi slli srli srai	$rd = rs1 \text{ op } imm$
01100111 OP	R	add sub sll slt sltu xor srl sra or and	$rd = rs1 \text{ op } rs2$
00011111 FENCE	I	fence	u nas: nop
11100111 SYSTEM	I	ecall, ebreak, csr*	u nas: nop

`funct3` wybiera wariant: rodzaj operacji ALU, szerokość load/store albo warunek skoku. `funct7` = 0100000 (czyli `instr[30] = 1`) odróżnia `sub` od `add` oraz `sra/srai` od `srl/srli`. Przy `slli/srli/srai` pole `rs2` zawiera `shamt`.

jalr z `rd == rs1` W `jalr a0, 0(a0)` cel liczysz ze *starej* wartości `a0`, a dopiero potem zapisujesz do niej adres powrotu. W C łatwo napisać to w złej kolejności. Test `06_jal_jalr` to sprawdza.

4.4 Pseudoinstrukcje i ładowanie stałych

Pseudo	Rozwinięcie	Pseudo	Rozwinięcie
<code>nop</code>	<code>addi x0, x0, 0</code>	<code>j L</code>	<code>jal x0, L</code>
<code>mv rd, rs</code>	<code>addi rd, rs, 0</code>	<code>call L</code>	<code>jal ra, L</code>
<code>not rd, rs</code>	<code>xori rd, rs, -1</code>	<code>ret</code>	<code>jalr x0, 0(ra)</code>
<code>neg rd, rs</code>	<code>sub rd, x0, rs</code>	<code>beqz rs, L</code>	<code>beq rs, x0, L</code>
<code>seqz rd, rs</code>	<code>sltiu rd, rs, 1</code>	<code>bgt a, b, L</code>	<code>blt b, a, L</code>

`li rd, imm32` rozwija się w `lui rd, hi + addi rd, rd, lo`. **Pułapka:** `addi` rozszerza 12-bitową stałą *znakiem*, więc dla `0x12345FFF` nie wystarczy `lui 0x12345; addi 0xFFF`, bo `0xFFF` to -1 . Poprawnie: $hi = (v + 0x800) \gg 12$, czyli `lui 0x12346; addi -1`.

4.5 Pamięć

Adresowanie jest bajtowe, **little-endian**: słowo 0x11223344 pod adresem 0 to bajty 44 33 22 11 pod adresami 0, 1, 2, 3. 1b/1h rozszerzają znakiem, a 1bu/1hu zerami. Zakładamy dostępy *wyrównane*.



Rysunek 4.2: Little-endian: najmłodszy bajt słowa pod najniższym adresem.

4.6 Nasza maszyna

To samo „środowisko” symuluje emulator (`emu/rv32sim.c`) i testbench RTL (`common/tb/soc_tb.sv`):

Adres	Co	Dostęp
0x0000_0000–0x0000_FFFF	RAM 64 KiB, tu ładowany jest program	R/W
0x1000_0000	TOHOST: zapis kończy symulację; 1 = PASS, ($n \ll 1$) 1 = FAIL w podteście n	W
0x1000_0004	PUTCHAR: wypisuje znak <code>wdata[7:0]</code>	W
0x1000_0008	CYCLES: licznik cykli (w emulatorze: liczba instrukcji)	R

Po resecie `pc = 0` i wszystkie rejestry mają wartość 0. Testy (`common/sw/tests/*.S`) trzymają w rejestrze `gp` numer podtestu i kończą skokiem do `pass` albo `fail` (`test_end.S`):

```
li    gp, 3           # podtest 3: slti / sltiu
slti  a1, a0, 6
li    t2, 1
bne   a1, t2, fail    # przy błędzie: TOHOST = (3 << 1) | 1
...
j     pass
.include "test_end.S"
```

4.7 Asembler i disassembler kursu

```
T=../common/tools/rvtool.py
python3 $T asm ../common/sw/tests/08_fib.S -I ../common/sw/tests -o fib.hex -l fib.lst
python3 $T dis fib.hex           # disasemblacja całego obrazu
python3 $T dis 0x00a50533        # jedno słowo: add a0, a0, a0
```

Format `.hex`: jedno 32-bitowe słowo na linię, od adresu 0 (to, co czyta `$readmemh`). Listing pokazuje adres, słowo, źródło i disasemblację.

4.8 Ślad wykonania

Emulator z opcją `-t` plik zapisuje po każdej instrukcji jedną linię:

```
00000010 00a50533 x10=0000002a      pc, słowo instrukcji, zapisany rejestr = wartość
00000014 fe051ce3 -                  instrukcja nie zapisała rejestru
```

Procesor w RTL wypisuje dokładnie ten sam format. Porównanie obu śladów (`tracecmp.py`) wskazuje *pierwszą* instrukcję, w której sprzęt odbiega od modelu. To najskuteczniejsza technika debugowania procesorów.

4.9 Emulator: struktura

Gotowe są pamięć z MMIO, ładowanie `.hex`, pętla główna i ślad. Ty piszesz funkcję `step()`:

```
// Wykonuje instrukcję spod c->pc. Zwraca numer zapisanego rejestru
// (0, jeśli instrukcja nic nie zapisała), wartość przez *rd_val.
static int step(cpu_t *c, uint32_t instr, uint32_t *rd_val) {
    uint32_t op = bits(instr, 6, 0);
    uint32_t rd = bits(instr, 11, 7);
    ...
    switch (op) {
        case 0x37: res = instr & 0xFFFFF000u; break;    // LUI (gotowe)
        // TODO: AUIPC, JAL, JALR, BRANCH, LOAD, STORE, OP-IMM, OP, FENCE, SYSTEM
    }
    if (writes && rd != 0) c->x[rd] = res;
    c->pc = next;
    ...
}
```

Wskazówki dla C: porównanie ze znakiem to `(int32_t)a < (int32_t)b`, przesunięcie arytmetyczne to `(uint32_t)((int32_t)a » n)` (formalnie *implementation-defined*, w praktyce działa wszędzie), a przesunięcie $o \geq 32$ to w C UB, więc maskuj: `b & 31`.

4.10 Zadania

Zadanie 4.1 — Kodowanie na kartce

Zakoduj ręcznie do postaci szesnastkowej i sprawdź `rvtool dis`: (1) `addi a0, a0, -1`, (2) `sw ra, 12(sp)`, (3) `srai a1, a2, 3`, (4) `lui t0, 0x10000`, (5) `beq a0, zero, +16`, (6) `jal ra, -2048`. Przy 5 i 6 rozpisz bity immediate'a na pozycje w instrukcji (rys. 4.1).

Zadanie 4.2 — Generator immediate'ów (`rtl/imm_gen.sv`)

make unit. Wektory (400 instrukcji wszystkich formatów) wygenerował asembler. Gdy test zgłosi błąd, zdekoduj instrukcję: `rvtool.py dis 0x....`

Zadanie 4.3 — Emulator (emu/rv32sim.c)

make emu-test uruchamia 12 programów, a make emu-test P=07_load_store jeden program ze śladem. Programy są ułożone od najprostszych (01_smoke potrzebuje tylko lui, addi, sw, jal), więc implementuj instrukcje w kolejności testów. Pisz z myślą o porównywaniu z RTL: ten emulator jest specyfikacją, więc jasność jest ważniejsza niż wydajność.

Zadanie 4.4 — Czytanie programu

make emu-test P=10_recursion, potem otwórz `build/hex/10_recursion.lst` i `build/10_recursion.trace`. Prześledź, jak sp maleje przy wywołaniach fib. Ile instrukcji wykonuje mul(1234, 5678) i dlaczego tyle? Ile razy wykonało się jalr i skąd ta liczba?

Moduł zaliczony, gdy...

make test → PASS dla imm_gen i 12 × PASS emulatora.

Pytania kontrolne

1. Który bit instrukcji jest zawsze bitem znaku immediate'a i dlaczego to ułatwia sprzęt?
2. Jaki jest zasięg beq i jal? Dlaczego bit 0 offsetu nie jest kodowany?
3. Rozwiń li a0, 0xDEADBEEF na lui i addi.
4. Co znajdzie się w rejestrze po lb z bajtu 0x80, a co po lbu?
5. Dlaczego emulator jest dobrym „wzorcem” dla RTL, choć sam może mieć błędy?

Rozdział 5

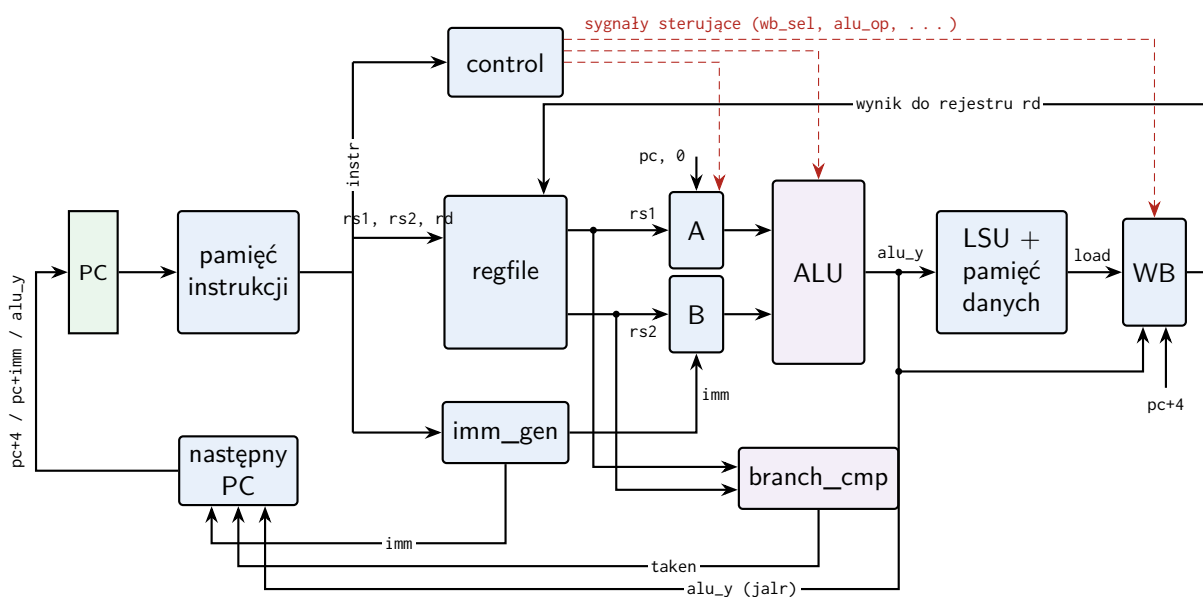
Procesor jednocyklowy RV32I

Cel: złożyć działający procesor, który wykonuje prawdziwe programy w kodzie maszynowym RISC-V, po jednej instrukcji na takt zegara.

Czas: 3–4 wieczory

Katalog: [05-single-cycle/](#)

5.1 Ścieżka danych



Rysunek 5.1: Ścieżka danych procesora jednocyklowego. Linie przerywane to sygnały sterujące z dekodera.

W jednym takcie, od zbrocza do zbrocza (rys. 5.1):

1. *pc* adresuje pamięć instrukcji, a *instr* pojawia się kombinacyjnie.
2. Dekoder (*control*) i *imm_gen* analizują *instr*, a bank rejestrów czyta *rs1* i *rs2*.
3. ALU liczy wynik, adres dla *load/store* albo cel skoku dla *jalr*.

4. Load: LSU wybiera bajty ze słowa z pamięci danych. Store: LSU ustawia wstrb/wdata, a zapis nastąpi na zboczu.
5. Multiplexer WB wybiera, co trafi do rd: wynik ALU, daną z pamięci albo pc+4. Zapis nastąpi na zboczu.
6. Logika następnego PC wybiera pc+4, pc+imm albo alu_y & ~1. Nowa wartość też trafia do PC na zboczu.

Wszystkie efekty instrukcji (zapis rejestru, zapis pamięci, nowe PC) dzieją się na tym samym zboczu zegara. Dlatego ten procesor jest taki prosty. Cena: okres zegara musi pomieścić całą drogę od PC przez pamięć instrukcji, rejestry, ALU i pamięć danych aż do zapisu.

5.2 Przejście instrukcji przez ścieżkę danych

Instrukcja	A	B	ALU	Do rd	Następne PC
add rd, rs1, rs2	rs1	rs2	{i[30], f3}	alu_y	pc+4
addi rd, rs1, imm	rs1	imm	{0, f3} [*]	alu_y	pc+4
lw rd, imm(rs1)	rs1	imm	ADD	load_data	pc+4
sw rs2, imm(rs1)	rs1	imm	ADD	— (zapis rs2)	pc+4
beq rs1, rs2, off	—	—	—	—	taken ? pc+imm : pc+4
jal rd, off	—	—	—	pc+4	pc+imm
jalr rd, imm(rs1)	rs1	imm	ADD	pc+4	alu_y & ~1
lui rd, imm	0	imm	ADD	alu_y	pc+4
auipc rd, imm	pc	imm	ADD	alu_y	pc+4

^{*} dla srari: {i[30], f3}

5.3 Tabela sterowania

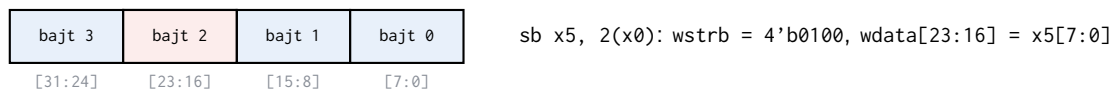
Wypełnij tę tabelę **na kartce**, zanim napiszesz control.sv. Kreska oznacza „obojętne”. Test tb_control sprawdza dokładnie tę tabelę na 97 zakodowanych instrukcjach i wypisuje różnice z nazwami pól.

	reg_write	wb_sel	alu_a_sel	alu_b_imm	alu_op	mem_read	mem_write	branch	jump	jump_reg
OP (R)										
OP-IMM										
LOAD										
STORE										
BRANCH										
JAL										
JALR										
LUI										
AUIPC										
FENCE/SYSTEM										

addi z ujemną stałą to nie sub Dla OP-IMM `instr[30]` jest częścią immediate'a. Gdyby dekodery brał go do `alu_op` jak dla R-type, `addi a0, a0, -1` (stała z ustawionym bitem 30) liczyłoby się jako odejmowanie. Bit 30 ma znaczenie tylko dla `srai` (`funct3 = 101`).

5.4 LSU: bajty w słowie

Pamięć jest zorganizowana w 32-bitowe słowa. `addr[31:2]` wybiera słowo, a `addr[1:0]` bajt w słowie. Zapis częściowy realizuje maska `wstrb` (*write strobe*): bit $i = 1$ oznacza zapis bajtu i .



Rysunek 5.2: Zapis bajtu pod adres o `addr[1:0] = 2`.

Przy odczycie pamięć zwraca całe słowo, a LSU przesuwa je o $8 \cdot \text{addr}[1:0]$ i rozszerza znakiem (1b, 1h) albo zerami (1bu, 1hu).

5.5 Interfejs rdzenia: kontrakt z testbenchem

```
module core import rv32i_pkg::*; (
  input logic      clk, rst,
  output logic [31:0] imem_addr, input logic [31:0] imem_rdata,
  output logic [31:0] dmem_addr, output logic dmem_re,
  input logic [31:0] dmem_rdata,
  output logic [3:0] dmem_wstrb, output logic [31:0] dmem_wdata,
  output logic      commit_valid,           // ślad wykonania
  output logic [31:0] commit_pc, commit_instr, commit_rd_val,
  output logic [4:0] commit_rd
```

```
);
```

- Pamięć (64 KiB) jest *jedna*, widziana przez dwa porty. Odczyt jest kombinacyjny, a zapis następuje na zboczu, gdy `dmem_wstrb != 0`.
- `dmem_addr` to pełny adres bajtowy, a `dmem_rdata` całe słowo. Wybór bajtów robi Twoje LSU.
- Reset synchroniczny, a po nim `pc = 0`.
- **Interfejs commit** służy tylko do weryfikacji. W każdym takcie, w którym kończy się instrukcja, `commit_valid = 1` i opis tej instrukcji. `commit_rd = 0`, gdy instrukcja nie zapisuje rejestru.

Testbench ([common/tb/soc_tb.sv](#)) przerywa symulację z czytelnym komunikatem, gdy: PC jest X po resecie, `commit_valid` jest X, następuje zapis pod adres X albo nieobsługiwany adres, wykona się słowo `0x00000000` (skok w pustą pamięć), przez 1000 taktów nie ma commitu albo przekroczony zostanie limit cykli.

5.6 Jak debugować procesor

1. Zaczynj od **najprostszego testu**, który nie przechodzi: `make prog P=01_smoke`.
2. **Ślad**. `make prog P=...` zapisuje `build/P.trace`. Porównaj go z emulatorem:

```
make -C ../04-isa-emulator emu-test P=05_branch
python3 ../common/tools/tracemp.py ../04-isa-emulator/build/05_branch.trace \
    build/05_branch.trace
```

Wynik pokazuje pierwszą różniącą się instrukcję z disasemblacją i odpowiedzią: inne PC oznacza błąd skoku, inna wartość błąd wykonania.

3. **Listing** `build/hex/P.lst` łączy adresy z liniami źródła.
4. **Przebiegi**: `make wave P=05_branch`. Dodaj `dut.pc`, `imem_rdata`, sygnały sterujące i `commit_*`.
5. „FAIL: podtest N” oznacza, że w źródle testu przed nieudanym sprawdzeniem stoi li gp, N.

Przykładowy raport `tracemp` dla błędu w dekodерze:

```
RÓŻNICA w instrukcji nr 12:
ostatnie zgodne:
  00000024: 00500593  addi a1, zero, 5          x11=00000005
oczekiwano (ref): 00000028: fff58593  addi a1, a1, -1    x11=00000004
jest      (RTL): 00000028: fff58593  addi a1, a1, -1    x11=00000006
-> ta sama instrukcja, inny efekt: błąd wykonania (ALU, LSU, forwarding...)
```

5.7 Programy testowe

01_smoke	tylko lui, addi, sw: najprostszy możliwy test
02_alu_imm, 03_alu_reg	ALU z immediate i rejestr-rejestr, przypadki brzegowe
04_lui_auiop	stałe górne i adresowanie względem PC
05_branch	wszystkie skoki warunkowe, także dalekie (>2 KiB)
06_jal_jalr	adresy powrotu, rd == rs1, stos, zagnieżdżone wywołania
07_load_store	wszystkie szerokości, rozszerzanie znakiem i zerami
08_fib, 09_sort, 10_recursion	„prawdziwe” programy: pętle, tablice, rekurencja
11_hazards	sekwencje groźne dla potoku (rozdział 8)
12_mmio	wypisuje napis przez PUTCHAR i czyta licznik cykli

Gdy przejdzie 12_mmio, zobaczysz w terminalu tekst wypisany przez Twój procesor.

5.8 Zadania

Zadanie 5.1 — Dekoder (rtl/control.sv)

```
make unit T=control.
```

Zadanie 5.2 — LSU (rtl/lsu.sv)

make unit T=lsu. Rozrysuj sobie słowo z czterema bajtami i zobacz, gdzie trafia sb pod addr[1:0] = 3.

Zadanie 5.3 — Rdzeń (rtl/core.sv)

Złóż ścieżkę danych z gotowych bloków. Moduły z rozdziałów 3 i 4 są dołączane z tamtych katalogów, więc poprawka w [alu.sv](#) od razu działa także tutaj. make progs uruchamia 12 programów, make prog P=08_fib jeden, a make test wszystko.

Zadanie 5.4 — Pomiary

make synth T=core: ile komórek ma Twój procesor w porównaniu z sumą ALU i banku rejestrów? CPI wynosi z definicji 1,000. W rozdziale 8 porównasz to z potokiem.

Uwaga o długości ścieżki Pamięci są w testbenchu, poza modulem core, więc Yosys widzi osobne kawałki: PC → imem_addr, imem_rdata → ... → dmem_addr oraz dmem_rdata → rejestry. Prawdziwy takt procesora jednocyklowego to *suma*: odczyt imem + dekodowanie + rejestry + ALU + odczyt dmem + zapis. To jest motywacja potoku.

Moduł zaliczony, gdy...

make test → 2 × PASS (testy jednostkowe) + 12 × PASS (programy), make lint czysto.

Pytania kontrolne

1. Wymień wszystkie elementy, przez które przechodzi 1w w ciągu jednego taktu.
2. Po co multiplexer A ma wejścia pc i 0? Które instrukcje ich używają?

3. Dlaczego cel jalr liczy ALU, a cel jal i beq osobny sumator?
4. Jakie wstrb i wdata wystawi LSU dla sh x7, 2(x0), gdy x7 = 0x1234BEEF?
5. Dlaczego procesor jednocyklowy ma niską maksymalną częstotliwość zegara?

Rozdział 6

Weryfikacja: co-symulacja, fuzzing, Verilator

Cel: przestać ufać zdaniu „przeszło 12 testów”. W prawdziwych projektach weryfikacja zajmuje więcej czasu niż projektowanie. Ten rozdział nie ma nowego RTL: tymi samymi narzędziami sprawdzasz rdzeń z rozdziału 5, a później potok z rozdziału 8.

Czas: 1–2 wieczory

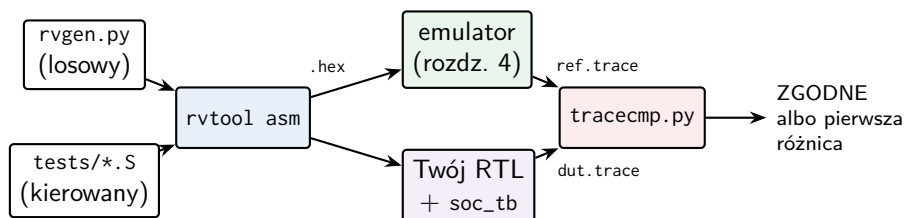
Katalog: [06-weryfikacja/](#)

6.1 Trzy poziomy testowania

Testy kierowane (*directed*): ręcznie napisane programy sprawdzające konkretne zachowania ([common/sw/tests/*.S](#)). Dobre na start i na znane przypadki brzegowe, słabe na te, o których nie pomyślałeś.

Co-symulacja z modelem referencyjnym: ten sam program wykonują emulator i RTL, a porównujemy *każdą* wykonaną instrukcję (PC, słowo, zapisany rejestr, wartość). Błąd zostaje wykryty tam, gdzie powstał, a nie 500 instrukcji później, gdy program „coś” źle policzy.

Testy losowe (*fuzzing*, w branży *constrained random*): generator tworzy tysiące programów, których nikt by ręcznie nie napisał, a co-symulacja ocenia, czy RTL zgadza się z modelem. Szczególnie skuteczne przy potoku: znajdują kombinacje zależności, skoków i loadów, o których nie wiedziałeś, że mogą być groźne.



Rysunek 6.1: Co-symulacja: ten sam program w dwóch niezależnych implementacjach.

Skąd model wie, co jest poprawne? Nie wie. Emulator też może mieć błąd. Siła metody polega na tym, że dwie *niezależne* implementacje (C i RTL, napisane różnymi technikami) rzadko mylą się tak samo. Gdy się różnią, jedna z nich ma błąd i rozstrzyga specyfikacja. Tak samo weryfikuje

się prawdziwe procesory: otwarte rdzenie RISC-V porównują ślad z modelami Spike albo Sail, a programy generuje m.in. riscv-dv.

6.2 Ograniczenia porównania śladów

- Ślad zawiera tylko zapisy do rejestrów. Błędny **store** wyjdzie dopiero przy późniejszym odczycie tego adresu. (Możliwe rozszerzenie: dodać do interfejsu commit adres i daną zapisu.)
- Odczyt licznika cykli (MMIO CYCLES) z natury daje różne wartości w emulatorze i w RTL. Dlatego make cosim pomija test 12_mmio, a ślady programów w C wywołujących cycles() rozjadają się w tym miejscu.
- Ślad RTL może być dłuższy od wzorcowego, bo testbench pozwala potokowi dokończyć instrukcje po zapisie do TOHOST. tracecmp porównuje długość śladu wzorcowego.

6.3 Verilator

Icarus to klasyczny symulator zdarzeniowy, który interpretuje opis i obsługuje cztery wartości logiczne. Verilator **kompiluje** RTL do C++, symuluje dwie wartości, myśli w cyklach zegara i jest zwykle 10–100× szybszy. Z tym samym testbenchem (opcja -timing obsługuje #5 i @(posedge)) program 10_recursion na rozwiązaniu wzorcowym wykonywał się w Icarusie ok. 0,55 s, a w Verilatorze ok. 0,01 s.

Verilator nie widzi X Niezainicjowany sygnał to dla Verilatora po prostu 0, więc błędy inicjalizacji są ukryte. Dobra praktyka: debuguj w Icarusie, testuj masowo w Verilatorze.

```
make progs SIM=verilator      # pierwsza kompilacja trwa kilka sekund
make fuzz N=500 SIM=verilator
```

6.4 Mutacje: kto pilnuje strażnika?

Skąd wiadomo, że testy są dobre? Wstaw do rdzenia *celowy* błąd i sprawdź, czy go wykryją. To się nazywa **testowanie mutacyjne**. Przy tworzeniu kursu w ten sposób odkryłem, że pierwsza wersja testów nie wykrywała błędów w bicie 11 immediate’a skoku warunkowego: żaden skok nie był dłuższy niż 2 KiB. Stąd „dalekie skoki” w 05_branch.S.

#	Mutant	progs	cosim	fuzz
1	sra działa jak srl			
2	lh nie rozszerza znakiem			
3	zapis do x0 nie jest ignorowany			
4	jalr nie zeruje bitu 0			
5	sb zawsze zapisuje bajt 0 słowa			
6	bit 11 B-immediate’a z instr[31] zamiast instr[7]			
7	sltiu porównuje ze znakiem			

6.5 Pokrycie funkcjonalne

„Ile testów przeszło” to zła miara. Lepiej pytać, *które zachowania zostały sprawdzone*. Czy każda instrukcja się wykonała? Czy bgeu był choć raz *wzięty*, a nie tylko niewzięty? Czy wystąpił load tuż po store pod ten sam adres? Tak myśli się o pokryciu (*coverage*). Narzędzia przemysłowe zbierają je automatycznie (covergroup w SystemVerilogu). W kursie policzysz je skryptem ze śladów.

6.6 Zadania

Polecenia uruchamiasz z katalogu [06-weryfikacja](#). Domyślnie testowany jest rdzeń z rozdziału 5, a z CORE=08 potok z rozdziału 8.

Zadanie 6.1 — Co-symulacja

make cosim wymaga działającego emulatora i rdzenia. Jeśli coś się rozjedzie, przeczytaj raport tracecmp i znajdź błąd. Pamiętaj, że może być także w emulatorze!

Zadanie 6.2 — Fuzzing

make fuzz (20 programów po 300 instrukcji), potem make fuzz N=300 LEN=600 SEED=1000. Przeczytaj [common/tools/rvgen.py](#) i jeden wygenerowany program ([build/core05/fuzz/r1.S](#)). Czego generator *nie* tworzy?

Zadanie 6.3 — Polowanie na mutanty

Wprowadzaj do rdzenia po jednym błędzie z tabeli powyżej i wypełnij ją: który mechanizm wykrył błąd? Dla mutanta, którego fuzzer nie wykrywa, rozszerz rvgen.py tak, żeby wykrywał (np. dalekie skoki: .space między skokiem a celem). Na koniec cofnij mutację!

Zadanie 6.4 — Pokrycie

Napisz krótki skrypt w Pythonie, który z plików [build/core05/fuzz/*.trace](#) policzy, ile razy wykonała się każda instrukcja (disassembler: from rvtool import dis). Czy wystąpiło wszystkie 37 instrukcji RV32I (bez fence/ecall/ebreak)?

Zadanie 6.5 — Lint i synteza całego rdzenia

make lint, make synth T=core.

Moduł zaliczony, gdy...

make cosim → wszystko ZGODNE, make fuzz N=200 → 0 różnic, a fuzzer po Twojej zmianie wykrywa mutanta nr 6.

Pytania kontrolne

1. Dlaczego co-symulacja lokalizuje błąd lepiej niż test, który na końcu sprawdza wynik?
2. Jakie błędy przejdą niezauważone przez porównanie śladu złożonego tylko z zapisów do rejestrów?
3. Kiedy wolisz Icarusa, a kiedy Verilatora?

4. Co mierzy testowanie mutacyjne i czym różni się od pokrycia kodu?

Rozdział 7

Programy w C na Twoim procesorze

Cel: uruchomić skompilowany kod C na procesorze, który sam zbudowałeś, bez systemu operacyjnego i bez libc. To terytorium, które znasz z pracy, tylko tym razem pod spodem jest Twój własny sprzęt.

Czas: 1–2 wieczory

Katalog: [07-c-bare-metal/](#)

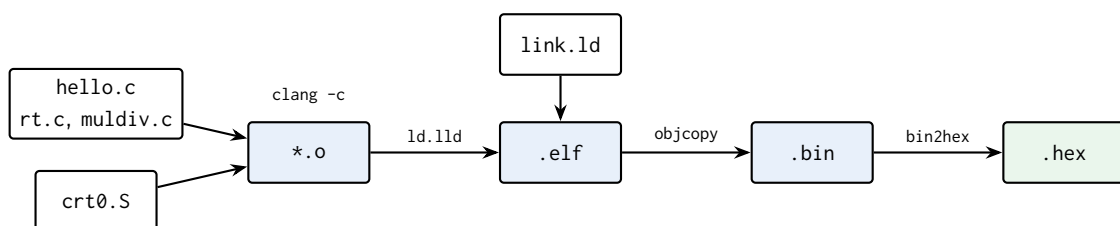
7.1 Toolchain

Makefile wykrywa toolchain sam. Wystarczy jedno z dwóch:

```
sudo apt install lld # clang już jest, brakuje tylko linkera
sudo apt install gcc-riscv64-unknown-elf # albo pełny toolchain GNU (umie też rv32)
```

make check-tools pokazuje, czego brakuje. Moduł został sprawdzony z clang 18 i ld.lld 18: wszystkie programy przechodzą na emulatorze, rdzeniu jednocyklowym i potoku. Ścieżka GCC jest w Makefile, ale nie była testowana.

7.2 Od C do pliku .hex



Rysunek 7.1: Kompilacja programu na nasz procesor.

Najważniejsze flagi:

- `-march=rv32i -mabi=ilp32`: tylko instrukcje bazowe; int, long i wskaźnik mają po 32 bity,
- `-ffreestanding -nostdlib`: nie ma libc ani standardowego kodu startowego,
- `-mno-relax`: linker nie zamienia adresowania na względne wobec gp (nasz crt0 nie ustawia gp),

- -O2: warto zobaczyć, jak dobry kod generuje kompilator (make dis P=bench).

7.3 Co się dzieje przed main()

Procesor po resecie zaczyna od `pc = 0` i nic nie wie o C. Kod startowy `crt0.S` musi:

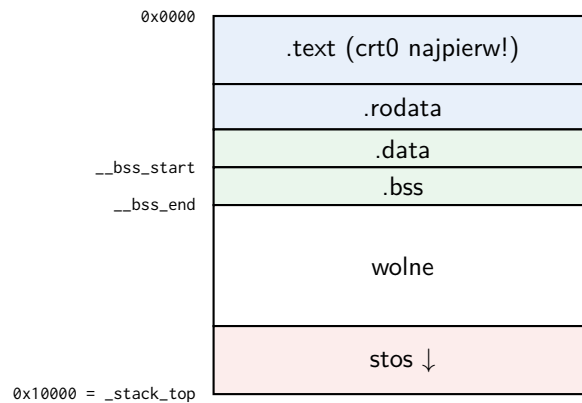
```

_start:
    la    sp, _stack_top      # 1. stos: C go potrzebuje do zmiennych lokalnych
    la    t0, __bss_start     # 2. zerowanie .bss: "static int x;" ma być 0
    la    t1, __bss_end
1:  bgeu  t0, t1, 2f
    sw    zero, 0(t0)
    addi  t0, t0, 4
    j     1b
2:  call  main                # 3. program
    ...                      # 4. a0 == 0 ? TOHOST = 1 : TOHOST = (a0 << 1) | 1

```

Sekcji `.data` nie trzeba kopiować, bo cały obraz programu ładuje w RAM przez `$readmemh`. Na prawdziwym mikrokontrolerze `.data` leży we flashu i `crt0` kopiuje ją do RAM.

7.4 Skrypt linkera



Rysunek 7.2: Układ pamięci programu (64 KiB RAM).

Symbole `__bss_start`, `__bss_end` i `_stack_top` definiuje `link.ld`, a używa ich `crt0.S`. W ten sposób linker przekazuje adresy do kodu.

7.5 Brakujące instrukcje: `__mulsi3` i spółka

RV32I nie ma mnożenia ani dzielenia. Gdy piszesz `a * b`, kompilator wstawia `call __mulsi3`. Zwykle dostarcza ją `libgcc` albo `compiler-rt`, a u nas piszesz ją sam (zadanie 7.2). Zasady:

- w `muldiv.c` **nie wolno** używać `*`, `/` ani `%`, bo kompilator wywołałby te same funkcje i powstałaby nieskończona rekurencja,

- dzielenie przez zero ma dawać to samo co instrukcje RISC-V M: $x/0 = 0xFFFFFFFF$, $x \bmod 0 = x$,
- dzielenie ze znakiem zaokrągla do zera: $-7/2 = -3$, $-7 \bmod 2 = -1$.

Mnożenie podaję jako gotowy przykład, a dzielenie piszesz sam. Mnożenie „pisemne” w systemie dwójkowym:

```
uint32_t __mulsi3(uint32_t a, uint32_t b) {
    uint32_t r = 0;
    while (b) {
        if (b & 1) r += a;    // bit mnożnika ustawiony: dodaj przesuniętą mnożną
        a <<= 1;
        b >>= 1;
    }
    return r;
}
```

Dzielenie robi się analogicznie (*restoring division*): przesuwaj resztę w lewo, dosuwaj kolejny bit dzielnej, a gdy reszta $\geq$ dzielnik, odejmij i ustaw bit ilorazu.

Kompilator potrafi też sam wstawić wywołania `memset` i `memcpy` (np. przy zerowaniu tablicy na stosie), nawet jeśli ich nie używasz. Dlatego są w `lib/rt.c`.

7.6 MMIO i volatile

```
#define MMIO_PUTCHAR ((volatile uint32_t *)0x10000004)
*MMIO_PUTCHAR = 'A';
```

Bez `volatile` kompilator mógłby uznać dwa zapisy pod ten sam adres za zbędne i usunąć jeden. Na tym poziomie nie ma różnicy między zmienną a rejestrem urządzenia: to ten sam sw, tylko pod inny adres, który testbench (albo prawdziwy dekodery adresów) kieruje do urządzenia.

7.7 Konwencja wywołań (ilp32)

Argumenty trafiają do `a0–a7`, wynik do `a0`, adres powrotu do `ra`. Rejestry `s0–s11` funkcja musi zachować, więc zapisuje je na stosie w prologu. `t*` i `a*` może niszczyć. Stos jest wyrównany do 16 bajtów. Typowy prolog i epilog:

```
addi sp, sp, -16
sw    ra, 12(sp)
sw    s0, 8(sp)
...
lw    s0, 8(sp)
lw    ra, 12(sp)
addi sp, sp, 16
ret
```

7.8 Zadania

Zadanie 7.1 — Hello, world

make (kompilacja), make emu P=hello (emulator), make run P=hello (Twój rdzeń; CORE=08 uruchamia potok). Pod jakim adresem leży counter z `hello.c` i w której jest sekcji? Co by się stało bez zerowania `.bss`? Ile instrukcji wykonuje hello, a ile z nich to crt0?

Zadanie 7.2 — lib/muldiv.c

make emu P=muldiv_test, potem make run P=muldiv_test. Wektory testowe obejmują dzielenie przez zero, INT_MIN / -1 i liczby ujemne.

Zadanie 7.3 — Benchmark

make run P=bench wypisuje wyniki i liczbę cykli dla CRC32, sita Eratostenesa, sortowania i mnożenia macierzy. Która część jest najdroższa? Sprofiluj ją na poziomie sprzętu:

```
cut -d' ' -f1 build/bench.trace | sort | uniq -c | sort -rn | head
```

i znajdź najczęstsze adresy w `build/bench.dis`.

Zadanie 7.4 — Własny program

Napisz `prog/coś.c`: liczby pierwsze do 10 000, drzewo BST w statycznej tablicy albo „grę w życie” wypisywaną znakami. main zwraca 0 jako PASS.

Moduł zaliczony, gdy...

make test → wszystkie PASS na emulatorze i na rdzeniu.

Pytania kontrolne

1. Co musi zrobić crt0, zanim wywoła main? Czego nie musi robić u nas, a musi na mikrokontrolerze?
2. Skąd crt0 zna adresy początku i końca `.bss`?
3. Dlaczego w `muldiv.c` nie można użyć operatora `*`?
4. Co się stanie, gdy usuniesz volatile z definicji MMIO_PUTCHAR i skompilujesz z `-O2`?

Rozdział 8

Procesor potokowy

Cel: przerobić procesor jednocyklowy na klasyczny 5-etapowy potok z obsługą wszystkich hazardów i zmierzyć, co to dało. Najtrudniejszy i najciekawszy rozdział kursu.

Czas: 4–6 wieczorów

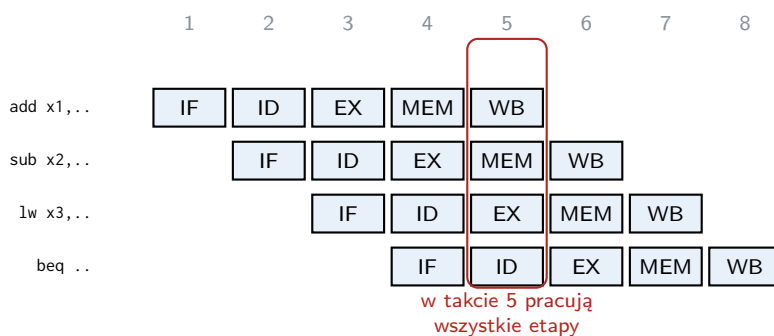
Katalog: [08-pipeline/](#)

8.1 Po co potok?

Czas wykonania programu („żelazne prawo” wydajności procesora):

$$\text{czas} = \text{liczba instrukcji} \times \text{CPI} \times T_{clk}.$$

Procesor jednocyklowy ma $\text{CPI} = 1$, ale bardzo długi takt. Potok tnie drogę instrukcji rejestrami na 5 krótszych etapów. Każda instrukcja nadal potrzebuje 5 etapów, więc *opóźnienie* pojedynczej instrukcji się nie zmienia. Ale **w każdym takcie kończy się jedna instrukcja**, a takt może być kilka razy krótszy.

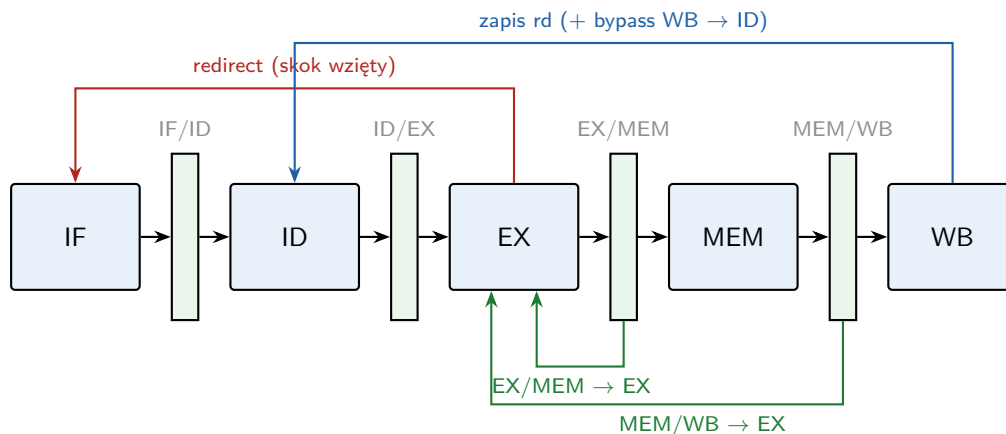


Rysunek 8.1: Idealny potok: od piątego taktu kończy się jedna instrukcja na takt.

Cena: instrukcje „zachodzą” na siebie i zależności między nimi trzeba obsłużyć w sprzęcie. To są **hazardy**.

8.2 Etapy i rejestry potoku

Etap	Co robi	Rejestr na końcu etapu
IF	imem_addr = pc, pobranie instrukcji, pc += 4	IF/ID: d_valid, d_pc, d_instr
ID	dekoder, immediate, odczyt rejestrów, wykrycie load-use	ID/EX: sterowanie, rs1_val, rs2_val, imm, numery rs1/rs2/rd
EX	forwarding, ALU, branch_cmp, cel skoku, redirect	EX/MEM: wynik, dana do store, rd
MEM	LSU i pamięć danych	MEM/WB: wartość do zapisu, rd
WB	zapis do banku rejestrów, commit	—

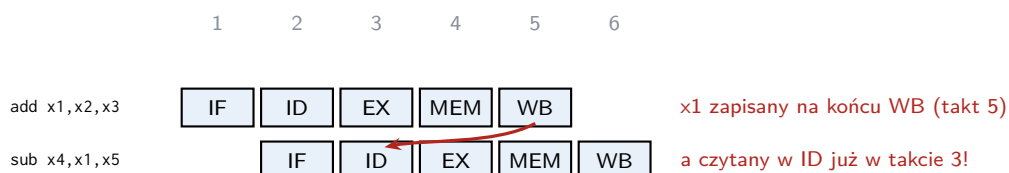


Rysunek 8.2: Pięć etapów, cztery rejestry potoku. Czerwony: redirect przy skoku. Zielone: ścieżki forwardingu. Niebieski: zapis do banku rejestrów.

Konwencje (zgodne ze szkieletem `rtl/core.sv`): przedrostek `d_` oznacza sygnał w ID, `e_` w EX, `m_` w MEM, a `w_` w WB. Każdy etap ma bit `*_valid`. **Bańka** (*bubble*) to etap z `valid = 0` i wyzerowanymi sygnałami, które mają skutki uboczne (`reg_write`, `mem_write`, `branch`, `jump` ...). Przepływa przez potok jak nop, ale nie jest raportowana jako commit.

Commit z WB. `commit_valid = w_valid`, a opis instrukcji (`pc`, `instr`) jedzie przez cały potok. Dzięki temu ślad jest identyczny jak w emulatorze i `make cosim` działa bez zmian.

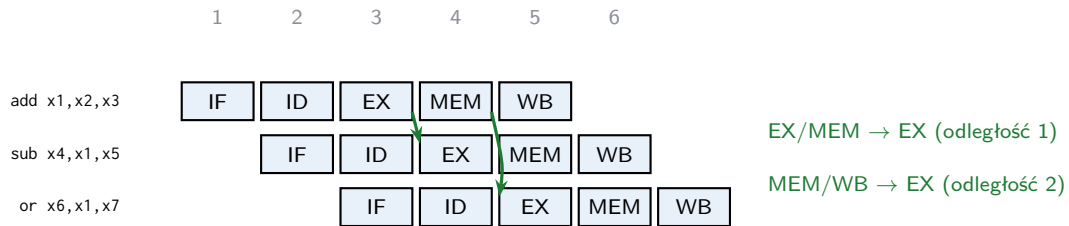
8.3 Hazardy danych



Rysunek 8.3: Hazard RAW (*read after write*).

Rozwiązania, od najprostszego:

1. **Programowo:** nop-y między zależnymi instrukcjami. Tak działa make progs PAD=n (rvtool -pad-nops) na etapie 8.1.
2. **Bypass WB → ID:** gdy WB w tym samym takcie zapisuje rejestr, który ID czyta, weź wartość prosto z WB (bank rejestrów oddałby starą).
3. **Forwarding do EX:** wynik add istnieje już na końcu EX. Następna instrukcja potrzebuje go na *początku* swojego EX, czyli takt później. Przekaż go z rejestru EX/MEM albo MEM/WB prosto na wejście ALU.
4. **Stall** przy *load-use* (niżej).

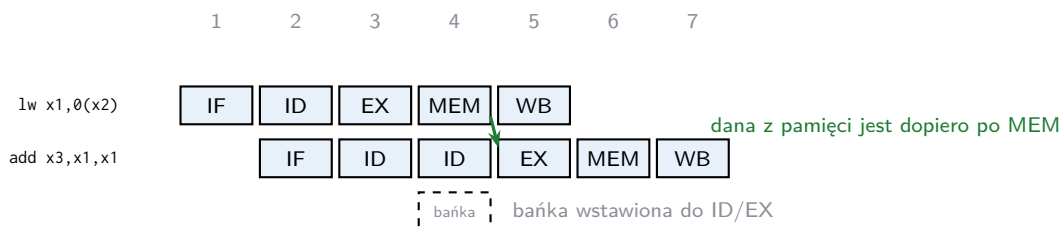


Rysunek 8.4: Forwarding: wynik z rejestru potoku wraca prosto na wejście ALU.

```
always_comb begin
    if (m_valid && m_reg_write && m_rd != 0 && m_rd == e_rs1) fwd_a = m_result; // najnowszy!
    else if (w_valid && w_reg_write && w_rd != 0 && w_rd == e_rs1) fwd_a = w_val;
    else fwd_a = e_rs1_val;
    // to samo dla fwd_b; fwd_b jest też daną dla store!
end
```

Trzy szczegóły forwardingu (1) **Kolejność:** gdy oba etapy piszą ten sam rejestr, wygrywa *nowszy*, czyli EX/MEM (test 11_hazards, podtest 4). (2) **rd != 0:** zapis do x0 nie może być forwardowany (podtest 8). (3) Dana do **store** (rs2) też musi przejść przez forwarding.

Load-use: jedyny przypadek, gdy trzeba czekać

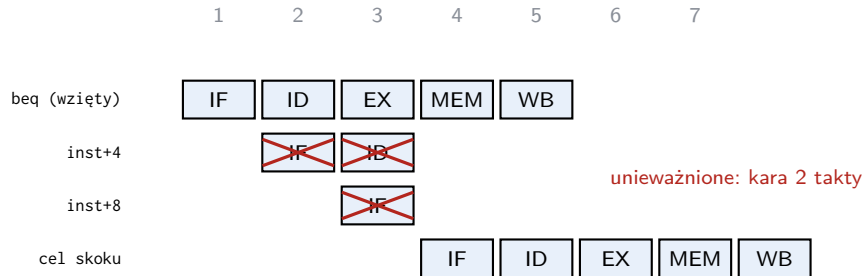


Rysunek 8.5: Load-use: instrukcja zależna czeka jeden takt w ID, a potem dostaje daną przez forwarding z MEM/WB.

Wykrywasz to w ID: w EX jest load, a jego rd to rs1 albo rs2 instrukcji w ID. Wtedy **PC i IF/ID** stoją, a do ID/EX idzie bańka.

8.4 Hazardy sterowania

Skok rozstrzyga się w EX. Do tego czasu IF i ID zdążyły pobrać dwie instrukcje „za skokiem”. Jeśli skok jest wzięty: pc = cel, a instrukcje w IF/ID i ID/EX trzeba **unieważnić** (*flush*).



Rysunek 8.6: Wzięty skok: dwie instrukcje pobrane „na ślepo” zamieniają się w bańki.

Strategia „zakładaj, że skok nie będzie wzięty” to najprostsza predykcja skoków. **Priorytety:** gdy w jednym takcie jest i redirect, i stall, wygrywa redirect, bo czekająca instrukcja i tak zostanie unieważniona.

8.5 Plan pracy

Szkielet [rtl/core.sv](#) ma sekcje dla etapów. Pracuj etapami i commituj po każdym, który działa.

Etap	Co dodajesz	Test
8.1	rejstry potoku, bity valid, redirect i flush przy skokach	make progs PAD=4
8.2	forwarding EX/MEM→EX i MEM/WB→EX (oba operandy i dana do store), bypass WB→ID	make progs PAD=1
8.3	stall load-use	make progs
8.4	weryfikacja	make cosim, make fuzz N=200
8.5	pomiary: CPI, synteza	make synth T=core

PAD=*n* wstawia *n* instrukcji nop *przed* każdą instrukcją. Etykiety wskazują na pierwszy z nich, żeby adresy powrotu jał się zgadzały. Przy PAD=4 żadna zależność danych nie jest groźna.

8.6 Zadania

Zadanie 8.1 — Potok bez obsługi hazardów danych

make progs PAD=4. Jaki jest najmniejszy PAD, przy którym Twój potok z etapu 8.1 przechodzi wszystkie testy? Wylicz to z diagramu potoku, zanim sprawdzisz. Jak zmienia się ta liczba po dodaniu samego bypassu WB→ID?

Zadanie 8.2 — Forwarding i bypass

make progs PAD=1. Dlaczego przy PAD=0 nadal coś pada? Które testy i dlaczego akurat te?

Zadanie 8.3 — Stall load-use

make progs: 12 × PASS bez żadnych nop-ów.

Zadanie 8.4 — Weryfikacja

make cosim, make fuzz N=200, a także make -C ../06-weryfikacja fuzz CORE=08 N=1000 SIM=verilator i make -C ../07-c-bare-metal test CORE=08. Fuzzer jest tu naprawdę cenny: losowe programy tworzą kombinacje zależności, skoków i load-ów, których nie ma w ręcznych testach.

Zadanie 8.5 — Pomiary

Porównaj CPI z make progs z rozdziałem 5 (zawsze 1,000). Na rozwiązaniu wzorcowym 09_sort ma CPI ok. 1,5, a bench z rozdziału 7 ok. 1,3. Ile taktów zabierają skoki, a ile load-use? Potem make synth T=core tu i w rozdziale 5. Który etap ma teraz najdłuższą ścieżkę? (Podpowiedź: EX, czyli forwarding, ALU, decyzja o skoku i multiplexer PC.)

Zadanie 8.6 — Dla ambitnych

Skoki rozstrzygane w ID (kara 1 takt, ale potrzebny forwarding do ID i nowy stall); statyczna predykcja BTFN (skoki wstecz zakładamy jako wzięte); predyktor dynamiczny z 2-bitowymi licznikami i BTB. Mierz CPI na bench.

Moduł zaliczony, gdy...

make progs → 12 × PASS przy PAD=0, make cosim → wszystko ZGODNE, make fuzz N=500 → 0 różnic, a bench z rozdziału 7 działa na CORE=08.

Pytania kontrolne

1. Dlaczego potok skraca czas wykonania programu, choć każda instrukcja trwa tyle samo albo dłużej?
2. Narysuj diagram potoku dla lw x1,0(x2); sw x1,4(x2). Czy potrzebny jest stall?
3. Dlaczego przy forwardingu EX/MEM ma pierwszeństwo przed MEM/WB?
4. Ile taktów traci potok na wziętym skoku i jak to zmniejszyć?
5. Co się stanie, jeśli przy stallu zapomnisz zatrzymać rejestr IF/ID (tylko PC)?

Rozdział 9

Co dalej: projekty rozszerzające

Cel:	wybrać kierunek dalszej pracy. Masz zweryfikowany procesor w dwóch mikroarchitekturach, emulator, infrastrukturę testową i toolchain C. Projekty poniżej są niezależne i wszystkie korzystają z narzędzi kursu.
Czas:	bez ograniczeń
Katalog:	09-dalej/

9.1 A. Rozszerzenie M (mnożenie i dzielenie)

Okolo 2 wieczorów. Instrukcje mul, mulh, mulhsu, mulhu, div, divu, rem, remu w sprzęcie i porównanie wydajności z programowym __mulsi3.

1. Emulator: przypadek funct7 = 0000001 dla opcode OP. Test: make -C ../04-isa-emulator emu-test-m. Uwaga na dzielenie przez zero i INT_MIN / -1.
2. RTL w wersji prostej: operator * w ALU (1 takt). make synth pokaże, ile kosztuje mnożarka.
3. Dzielenie iteracyjne w 32 taktach: potok musi czekać (stall z EX), aż dzielnik skończy. To dobre ćwiczenie z wielocyklowych jednostek wykonawczych.
4. Testy: [common/sw/tests_m/m_ext.S](#), make fuzz FUZZ_FLAGS=-mext, C z ARCH=rv32im, porównanie cykli bench.

9.2 B. CSR, wyjątki i przerwania

Okolo 4-6 wieczorów. Celem jest procesor, na którym da się uruchomić RTOS.

1. Zicrs: csrrw/csrrs/csrrc i rejestry mcycle, minstret, mstatus, mtvec, mepc, mcause, mie, mip.
2. Wyjątki: ecall, ebreak, nielegalna instrukcja (sygnał illegal z dekodera już jest!), niewyrównany dostęp. Skok do mtvec, powrót przez mret.
3. W potoku wyjątek musi być **precyzyjny**: starsze instrukcje się kończą, a młodsze zostają unieważnione.
4. Przerwanie od timera (mtime/mtimecmp jako MMIO).

9.3 C. Procesor na FPGA

Okolo 3–5 wieczorów i płytka. Program w C wypisuje tekst z Twojego procesora przez UART do terminala na komputerze.

Płytki z w pełni otwartym toolchainem (Yosys + nextpnr): **iCEBreaker** lub **iCESugar** (iCE40UP5K), **Tang Nano 9K/20K** (Gowin, Apicula), **ULX3S** lub **OrangeCrab** (ECP5).

Co trzeba zmienić:

- **Pamięć:** blokowy RAM w FPGA ma odczyt *synchroniczny*. Procesor jednocyklowy tego wprost nie zniesie, więc potrzebny jest odczyt na zboczu opadającym, wersja wielocyklowa albo potok z adresem imem podawanym takt wcześniej.
- **UART:** uart_tx z rozdziału 2 jako urządzenie MMIO z bitem busy do odczytu.
- **Zegar i reset:** PLL albo dzielnik, reset z przycisku przez synchronizator.
- **Timing:** nextpnr poda prawdziwą maksymalną częstotliwość, czyli rzetelne porównanie procesora jednocyklowego z potokiem.

9.4 D. Pamięć z opóźnieniem i cache

Okolo 4 wieczorów. Interfejs req/valid/ready zamiast „magicznej” pamięci, stall całego potoku przy chybieniu, cache instrukcji direct-mapped z liniami 16 B i pomiar trafień na bench.

9.5 E. Weryfikacja formalna

Okolo 2–3 wieczorów. **riscv-formal** z SymbiYosys dowodzi zgodności z ISA. Rdzeń wystawia interfejs **RVFI**, czyli rozszerzoną wersję Twojego commit_*. Na początek wystarczą prostsze asercje SVA: „bańka nigdy nie ma mem_write”, „po stallu w ID jest ta sama instrukcja”.

9.6 F. Oficjalne testy riscv-tests

Okolo 1 wieczoru. Testy rv32ui-p-* z repozytorium riscv-software-src/riscv-tests używają tego samego protokołu TOHOST co my. Wymagają toolchainu GNU i dostosowania skryptu linkera do naszej mapy pamięci.

9.7 Lektura

- **Harris & Harris**, *Digital Design and Computer Architecture: RISC-V Edition*. Rozdziały 5, 7 i 8 to ścieżka tego kursu z innej perspektywy.
- **Patterson & Hennessy**, *Computer Organization and Design, RISC-V Edition*. Klasyka, rozdział 4 o potoku.
- **Hennessy & Patterson**, *Computer Architecture: A Quantitative Approach*. Następny poziom: wykonanie poza kolejnością, predykcja, pamięć.

- **The RISC-V Instruction Set Manual**, tomy I (Unprivileged) i II (Privileged). Zaskakująco czytelne, z uzasadnieniami decyzji projektowych.
- **Sutherland, Davidson, Flake**, *System Verilog for Design*.

9.8 Prawdziwe rdzenie do przeczytania

PicoRV32	jeden plik, wielocyklowy, nastawiony na mały rozmiar
SERV	bit-serialny: 1 bit na takt, najmniejszy rdzeń RISC-V
Ibex (lowRISC)	2–3 etapy, jakość produkcyjna, świetna dokumentacja weryfikacji
VexRiscv	konfigurowalny potok w SpinalHDL
CVA6	64-bitowy, 6 etapów, uruchamia Linuksa

Porównaj ich decyzje ze swoimi: gdzie rozstrzygają skoki, jak zbudowany jest bank rejestrów i jak obsługują pamięć.

Dodatek A

Karta RV32I

Kodowanie wszystkich 37 instrukcji obliczeniowych, pamięciowych i sterujących RV32I (bez fence, ecall, ebreak). Tabela jest wygenerowana z tablic asemblera kursu (common/tools/rvtool.py), który koduje instrukcje bit w bit tak samo jak LLVM. Formaty i immediate'y: rys. 4.1.

Instr.	Fmt	opcode	funct3	funct7	Działanie
lui	U	0110111	–	–	rd = imm $\ll$ 12
auipc	U	0010111	–	–	rd = pc + (imm $\ll$ 12)
jal	J	1101111	–	–	rd = pc+4; pc += imm
jalr	I	1100111	000	–	rd = pc+4; pc = (rs1+imm) & ~1
beq	B	1100011	000	–	rs1 == rs2
bne	B	1100011	001	–	rs1 != rs2
blt	B	1100011	100	–	rs1 $<_s$ rs2
bge	B	1100011	101	–	rs1 $\geq_s$ rs2
bltu	B	1100011	110	–	rs1 $<_u$ rs2
bgeu	B	1100011	111	–	rs1 $\geq_u$ rs2
lb	I	0000011	000	–	rd = sext(M[rs1+imm][7:0])
lh	I	0000011	001	–	rd = sext(M[rs1+imm][15:0])
lw	I	0000011	010	–	rd = M[rs1+imm]
lbu	I	0000011	100	–	rd = zext(M[rs1+imm][7:0])
lhu	I	0000011	101	–	rd = zext(M[rs1+imm][15:0])
sb	S	0100011	000	–	M[rs1+imm][7:0] = rs2[7:0]
sh	S	0100011	001	–	M[rs1+imm][15:0] = rs2[15:0]
sw	S	0100011	010	–	M[rs1+imm] = rs2
addi	I	0010011	000	–	rd = rs1 + imm
slti	I	0010011	010	–	rd = rs1 $<_s$ imm
sltiu	I	0010011	011	–	rd = rs1 $<_u$ imm
xori	I	0010011	100	–	rd = rs1 ^ imm
ori	I	0010011	110	–	rd = rs1 imm
andi	I	0010011	111	–	rd = rs1 & imm
slli	I*	0010011	001	0000000	rd = rs1 $\ll$ shamt
srli	I*	0010011	101	0000000	rd = rs1 $\gg_u$ shamt
srai	I*	0010011	101	0100000	rd = rs1 $\gg_s$ shamt
add	R	0110011	000	0000000	rd = rs1 + rs2
sub	R	0110011	000	0100000	rd = rs1 - rs2
sll	R	0110011	001	0000000	rd = rs1 $\ll$ rs2[4:0]
slt	R	0110011	010	0000000	rd = rs1 $<_s$ rs2
sltu	R	0110011	011	0000000	rd = rs1 $<_u$ rs2
xor	R	0110011	100	0000000	rd = rs1 ^ rs2
srl	R	0110011	101	0000000	rd = rs1 $\gg_u$ rs2[4:0]
sra	R	0110011	101	0100000	rd = rs1 $\gg_s$ rs2[4:0]
or	R	0110011	110	0000000	rd = rs1 rs2
and	R	0110011	111	0000000	rd = rs1 & rs2

I* — format I, w którym `imm[11:5]` pełni rolę `funct7`, a `imm[4:0]` to `shamt`. Wszystkie immediate'y poza U są rozszerzane znakiem. `sext/zext`: rozszerzenie znakiem/zeraami.

Mapa pamięci maszyny kursu

<code>0x0000_0000–0x0000_FFFF</code>	RAM 64 KiB	program, dane, stos
<code>0x1000_0000</code>	TOHOST (W)	1 = PASS, ($n \ll 1$) 1 = FAIL n
<code>0x1000_0004</code>	PUTCHAR (W)	znak <code>wdata[7:0]</code>
<code>0x1000_0008</code>	CYCLES (R)	licznik cykli

Dodatek B

Ściągą SystemVerilog

Składnia

```
module nazwa import rv32i_pkg::*; #(parameter int W = 8) (  
    input logic      clk, rst,  
    input logic [W-1:0] a,  
    output logic [W-1:0] y  
);  
    logic [W-1:0] t;          // sygnał wewnętrzny  
    localparam logic [3:0] STALA = 4'hA; // stała lokalna  
  
    assign t = a ^ {W{1'b1}}; // logika kombinacyjna (krótka)  
  
    always_comb begin        // logika kombinacyjna (dłuższa)  
        y = '0;              // wartość domyślna: brak zatrzasku  
        case (t[1:0])  
            2'd0: y = a;  
            default: y = t;  
        endcase  
    end  
  
    always_ff @(posedge clk) // rejestr  
        if (rst) q <= '0;  
        else    q <= y;  
endmodule
```

Literały i operatory

8'hFF, 4'b1010, 32'd7, '0, '1	literały z rozmiarem; same zera/jedynki
x[7:0], x[8*i +: 8]	wycinek stały; wycinek o zmiennym początku
{a, b}, {4{b}}	sklejanie; powielanie
& ^ ~	bitowe; jako unarne: redukcja (1x = „jakikolwiek bit”)
&& !	logiczne (1 bit)
== !=, === !==	równość; dosłowna (z X i Z), tylko w testbenchach
< > >>	przesunięcia; >> arytmetyczne tylko dla signed
\$signed(x), \$unsigned(x)	interpretacja ze znakiem / bez znaku
W'(x)	rzutowanie na szerokość W
\$clog2(N)	$\lceil \log_2 N \rceil$ (szerokość adresu dla N elementów)

Zasady kursu

1. `always_comb` + =, `always_ff` + <=. Nigdy odwrotnie.
2. W `always_comb` każdy sygnał ma wartość domyślną na początku bloku.
3. Każdy case ma default.
4. Jeden zegar, reset synchroniczny aktywny w stanie 1.
5. Stałe z pakietu, nie „magiczne liczby”.
6. Uważaj na szerokości: `make lint` musi być czysty.

Testbench

```
`timescale 1ns/1ps
module tb_x;
    `include "tb_common.svh"
    logic clk = 0;
    always #5 clk = ~clk;
    initial begin
        tb_start();
        @(negedge clk);                // zmieniaj wejścia na zboczu opadającym
        `CHECK_EQ(y, 8'h42, "opis")
        repeat (10) @(negedge clk);
        $display("a=%h b=%0d", a, b);
        tb_finish();
    end
endmodule
```

Dodatek C

Ściąga poleceń

Polecenie	Działanie
make test	wszystkie testy modułu
make unit [T=nazwa]	testy jednostkowe (tb/tb_nazwa.sv)
make wave T=nazwa	test + przebiegi w GTKWave
make lint	Verilator -lint-only -Wall
make synth T=moduł	Yosys: komórki i najdłuższa ścieżka
make progs [PAD=n] [SIM=verilator]	12 programów testowych na rdzeniu
make prog P=08_fib	jeden program: wyjście, ślad build/P.trace , listing
make wave P=08_fib	przebiegi wykonania programu
make cosim	porównanie śladu z emulatorem
make fuzz N=100 [LEN=300 SEED=1]	losowe programy + porównanie
make emu-test [P=...]	(moduł 04) programy na emulatorze
make run P=hello [CORE=08]	(moduł 07) program w C na rdzeniu
make clean	usuwa build/

Narzędzia w common/tools

```
python3 rvtool.py asm prog.S -o prog.hex -l prog.lst [-I katalog] [--pad-nops N]
python3 rvtool.py dis prog.hex          # albo: dis 0x00a50533
python3 rvtool.py bin2hex prog.bin -o prog.hex
python3 tracecmp.py ref.trace dut.trace
python3 rvgen.py --seed 7 -n 300 -o rand.S [--mext]
```

Dodatek D

Słowniczek

Polski	Angielski	Znaczenie
przerzutnik	flip-flop (FF)	element pamiętający bit na zboczu zegara
zatrask	latch	element pamiętający sterowany poziomem; w RTL zwykle błąd
ścieżka krytyczna	critical path	najdłuższa droga kombinacyjna między rejestrami
bank rejestrów	register file	32 rejestry z portami odczytu i zapisu
ścieżka danych	datapath	bloki przetwarzające dane (ALU, rejestry, multipleksery)
jednostka sterująca	control unit	dekoder wytwarzający sygnały sterujące
potok	pipeline	podział wykonania na etapy oddzielone rejestrami
hazard	hazard	sytuacja, w której potok dałby zły wynik bez dodatkowej logiki
przekazywanie	forwarding / bypass	dostarczenie wyniku prosto z rejestru potoku
wstrzymanie	stall	zatrzymanie części potoku na takt
bańka	bubble	pusty etap potoku ($valid = 0$)
unieważnienie	flush	zamiana instrukcji z błędnej ścieżki w bańki
zatwierdzenie	commit / retire	zakończenie instrukcji, widoczne w śladzie
stała natychmiastowa	immediate	stała zakodowana w instrukcji
rozszerzenie znakiem	sign extension	kopiowanie bitu znaku na starsze bity
co-symulacja	co-simulation	porównywanie RTL z modelem w trakcie wykonania
model referencyjny	golden model	niezależna implementacja traktowana jako wzorzec
pokrycie	coverage	miara tego, które zachowania zostały sprawdzone